

Plan du cours

- Rappel sur les notions de bases en machine learning
- Apprentissage non supervisé
 - Clustering (hiérarchique, KMeans)
 - Réduction de dimensionnalité (PCA)
- Apprentissage supervisé
 - Toujours plus d'algorithmes
 - Gradient boosting
 - Réseaux de neurones
 - Feature selection
 - Explicabilité

Apprentissage supervisé

Gradient boosting & ANN

Feature selection & Explicabilité

Adaptive & Gradient Boosting

Rappel

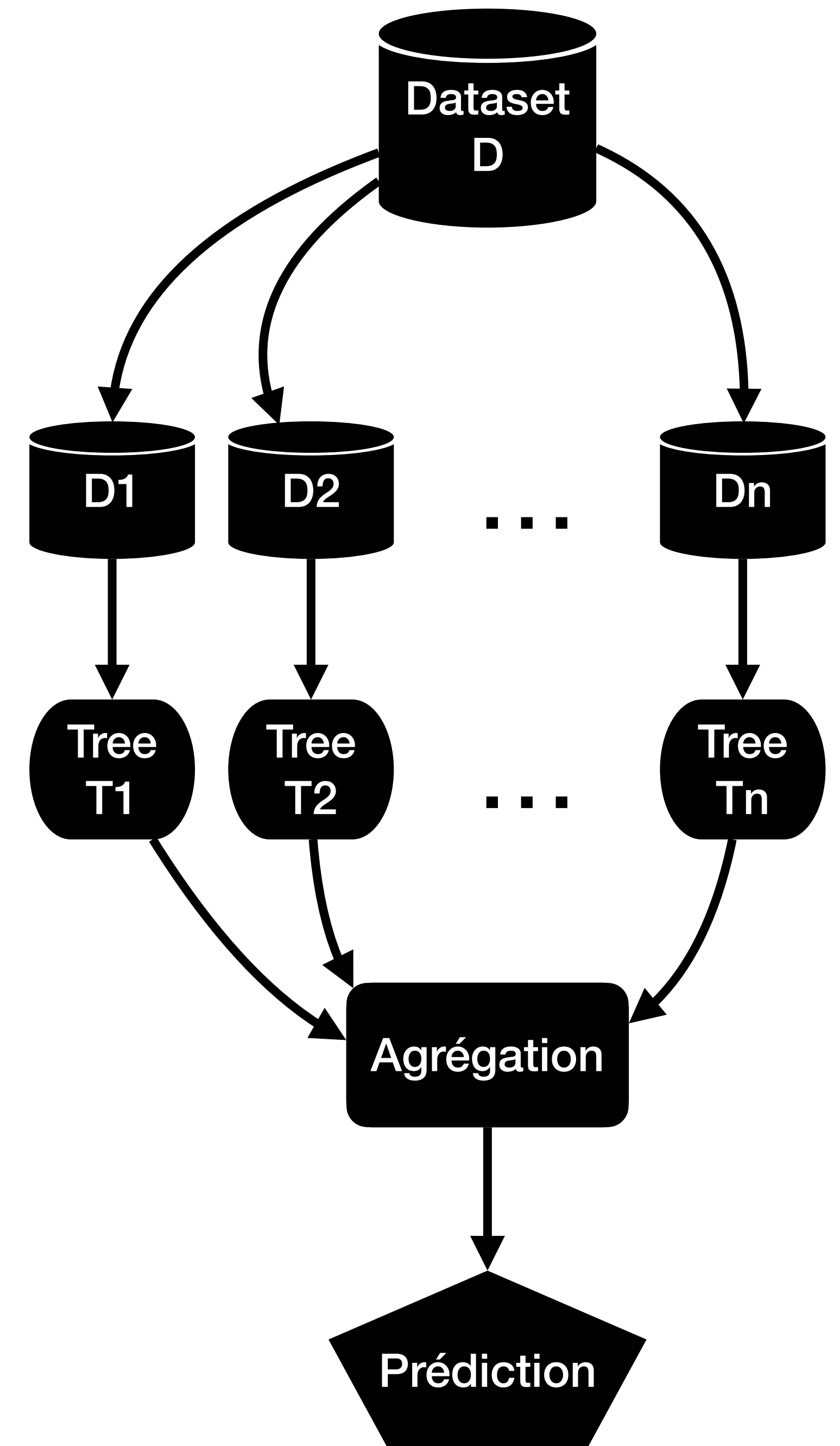
Méthode ensembliste - Ensemble learning

- Utilisation de plusieurs estimateurs utilisant une sous-partie des données/
caractéristiques
- Agrégation des prédictions de chacun des estimateurs
 - Moyenne pour la régression
 - Vote pour la classification

Rappel

Random Forest

- Entraînement d'un ensemble d'arbres de décision
- Utilisation d'un sous-ensemble aléatoire
 - des observations/samples (bagging)
 - des caractéristiques/features (feature sampling)
- Arbres tous entraînés indépendamment



Adaptive Boosting

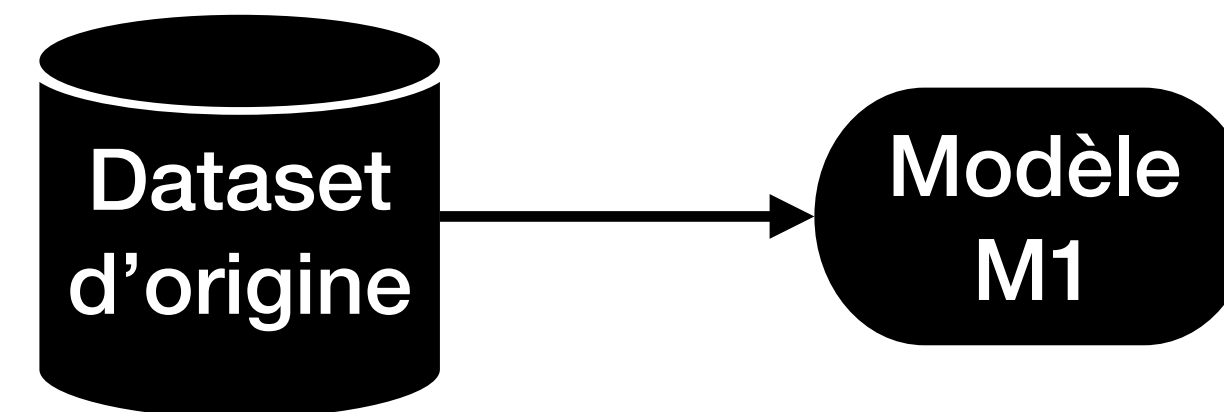
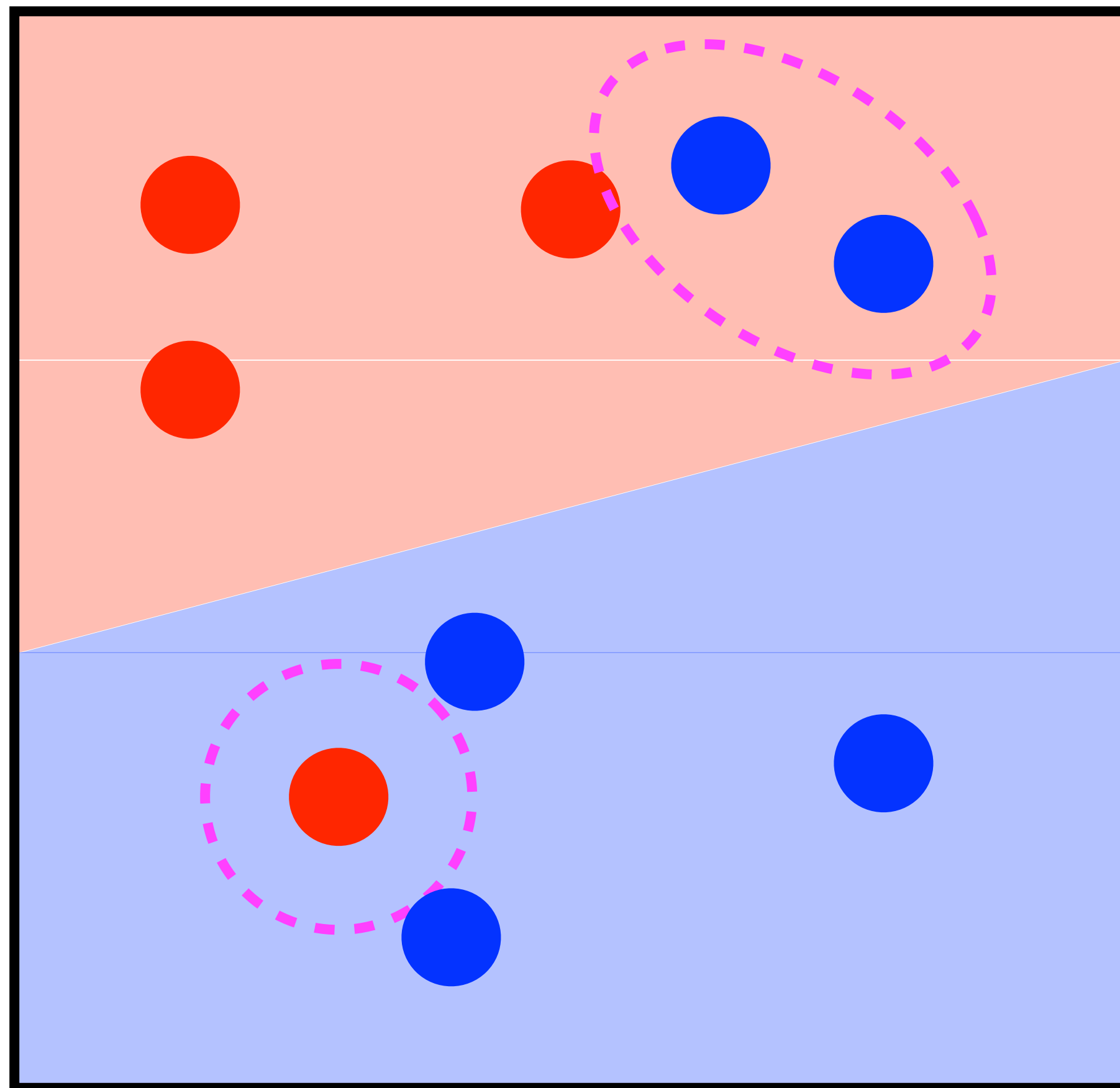
Adaptive Boosting (AdaBoost)

Principe

- Ensemble learning
- Utilisation de multiples estimateurs simples (sous-apprentissage)
- Apprentissage en cascade en informant les apprentissages avec les erreurs passées
- Poids de chaque observation évolue au cours de chaque étape de l'apprentissage
- Combinaison pondérée des décisions prises par chaque estimateur
- AdaBoost: utilisation d'arbre de décision comme modèle

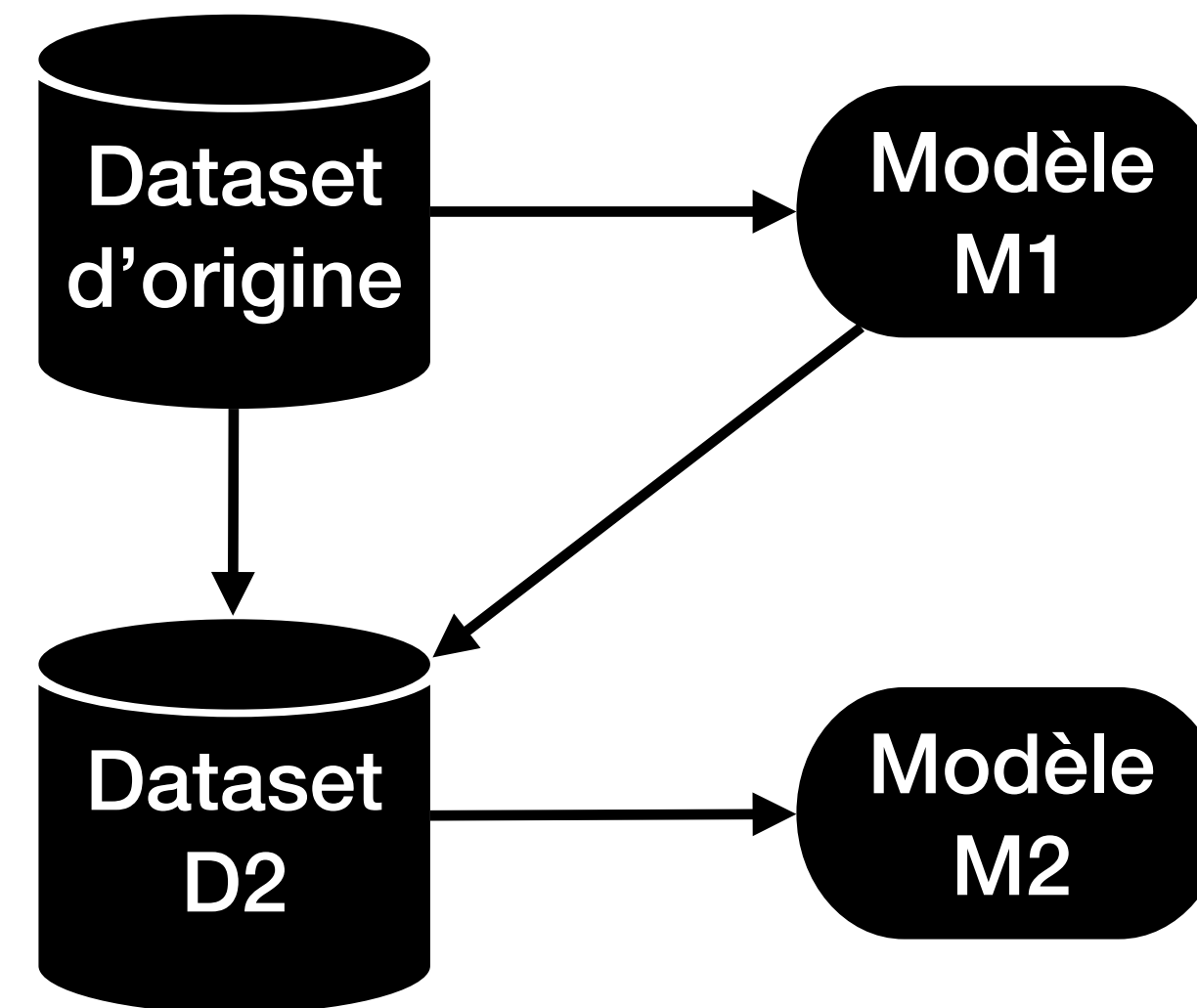
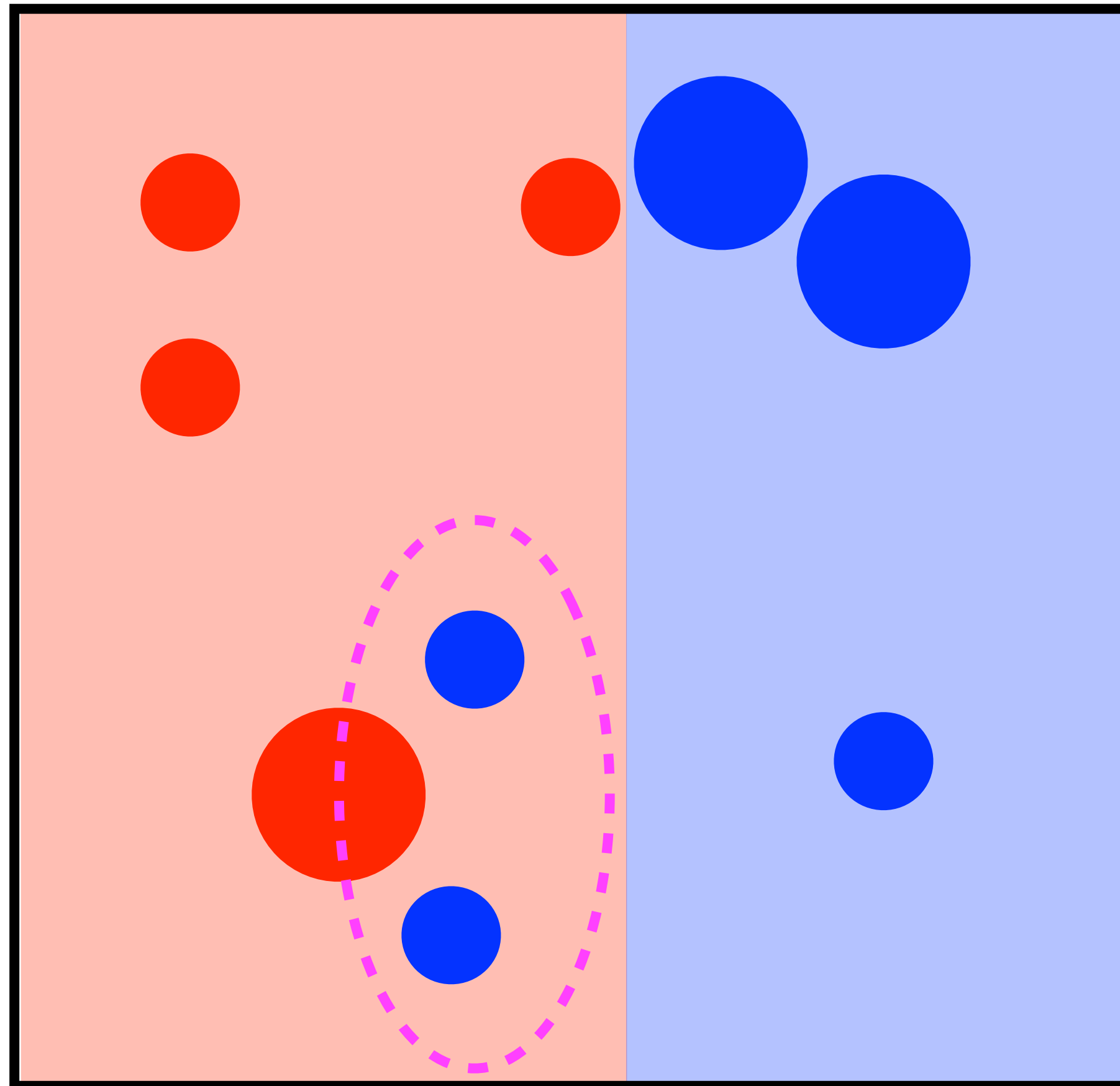
Adaptive Boosting (AdaBoost)

Exemple



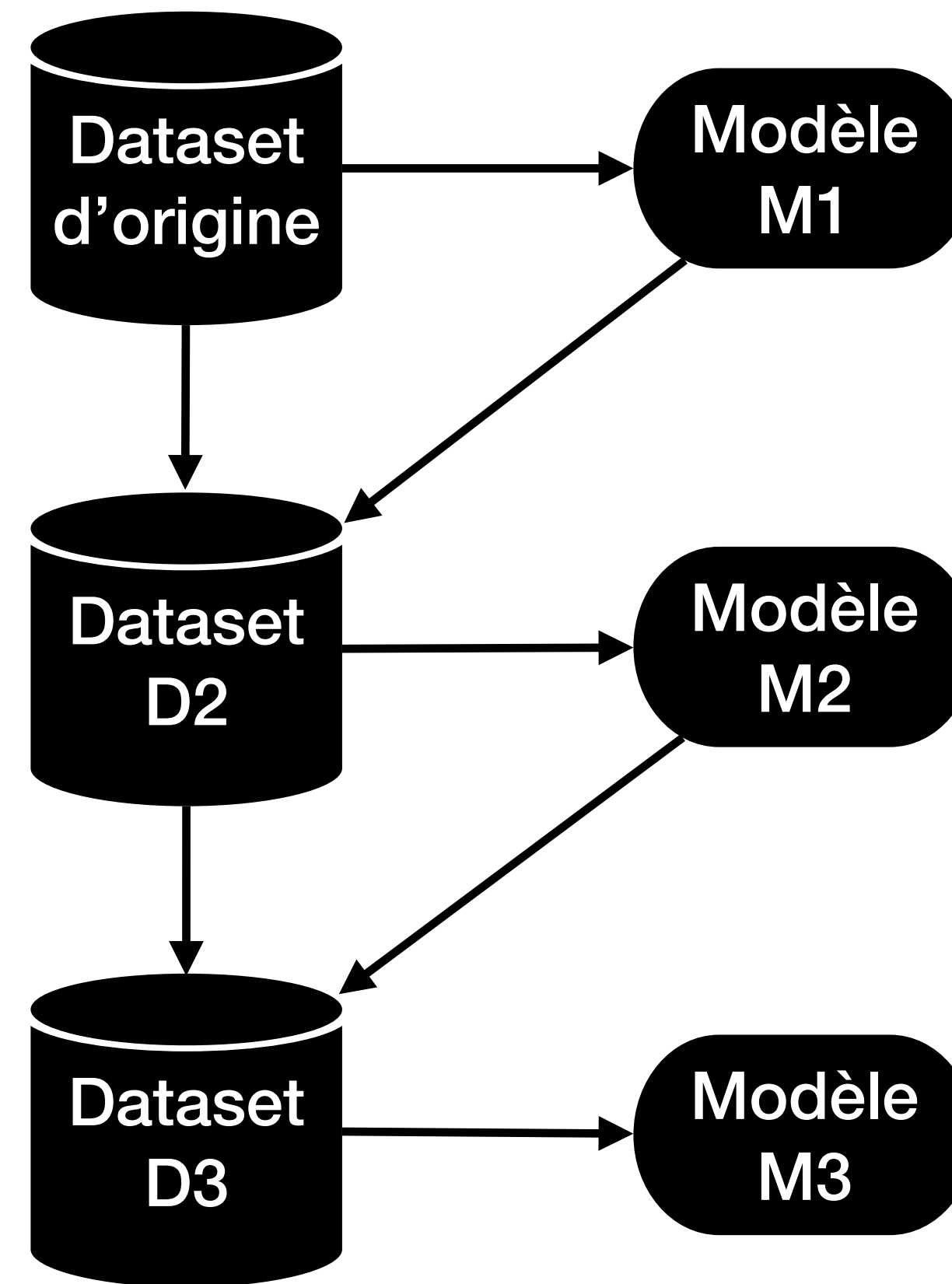
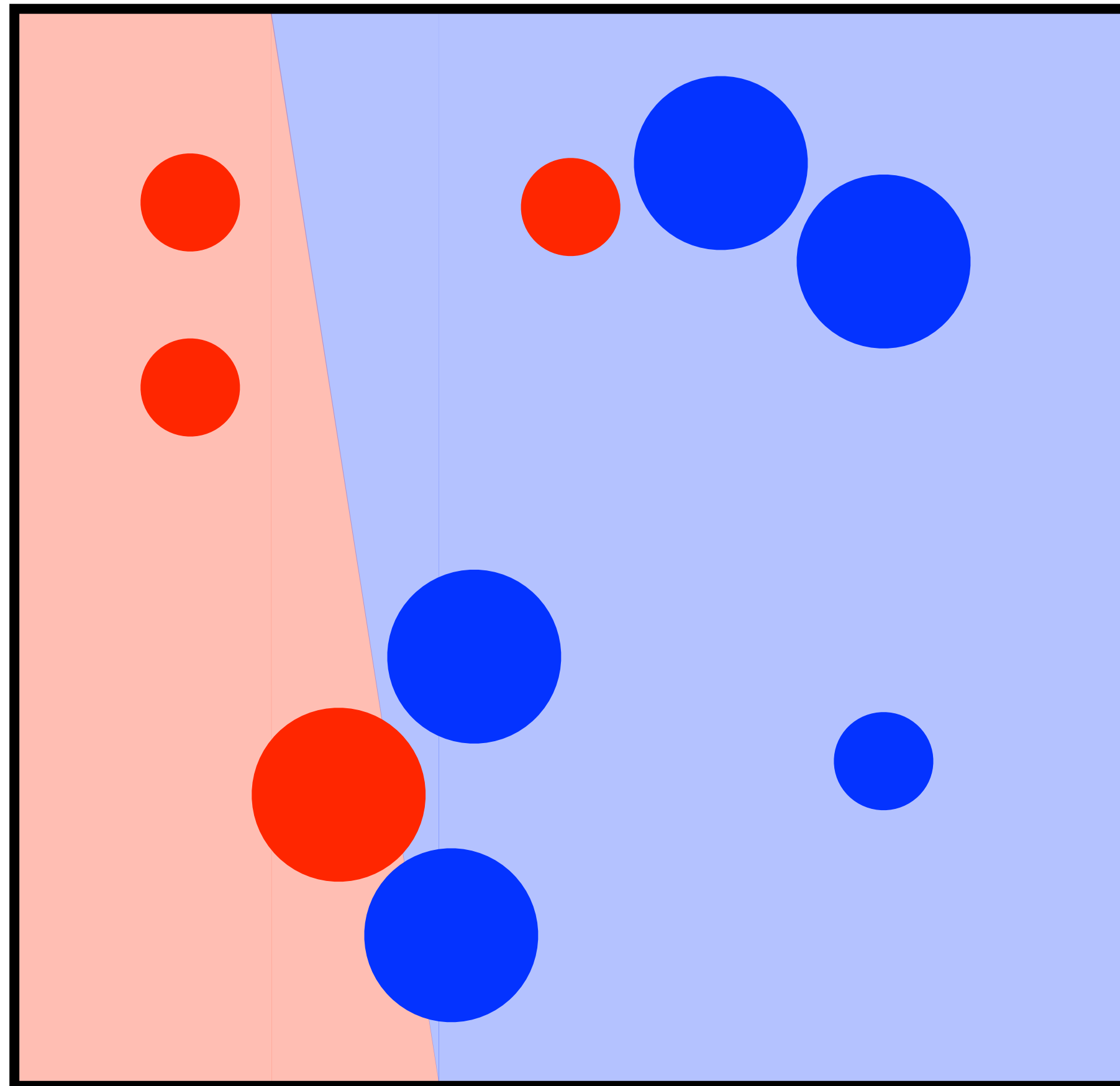
Adaptive Boosting (AdaBoost)

Exemple



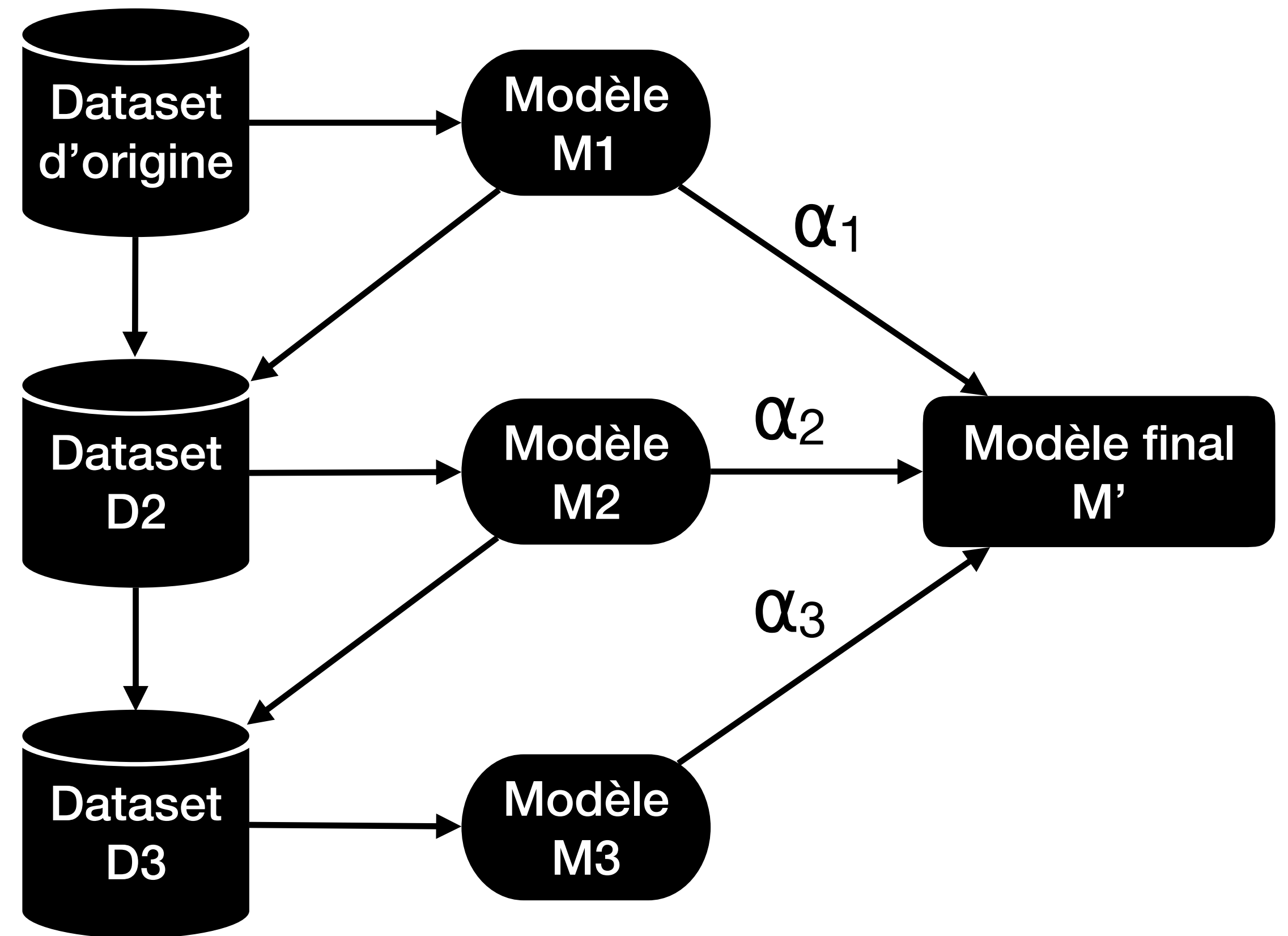
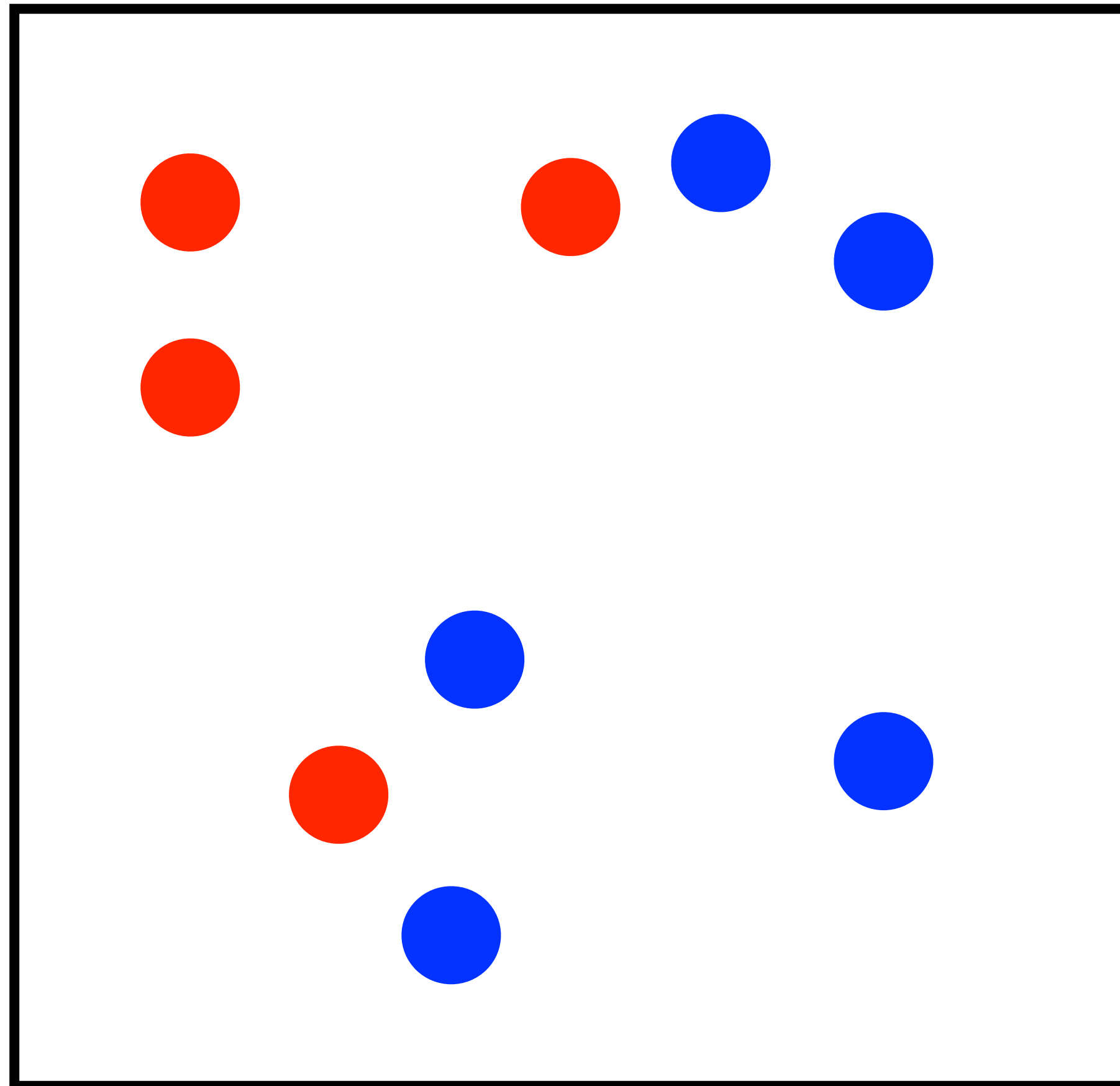
Adaptive Boosting (AdaBoost)

Exemple



Adaptive Boosting (AdaBoost)

Exemple

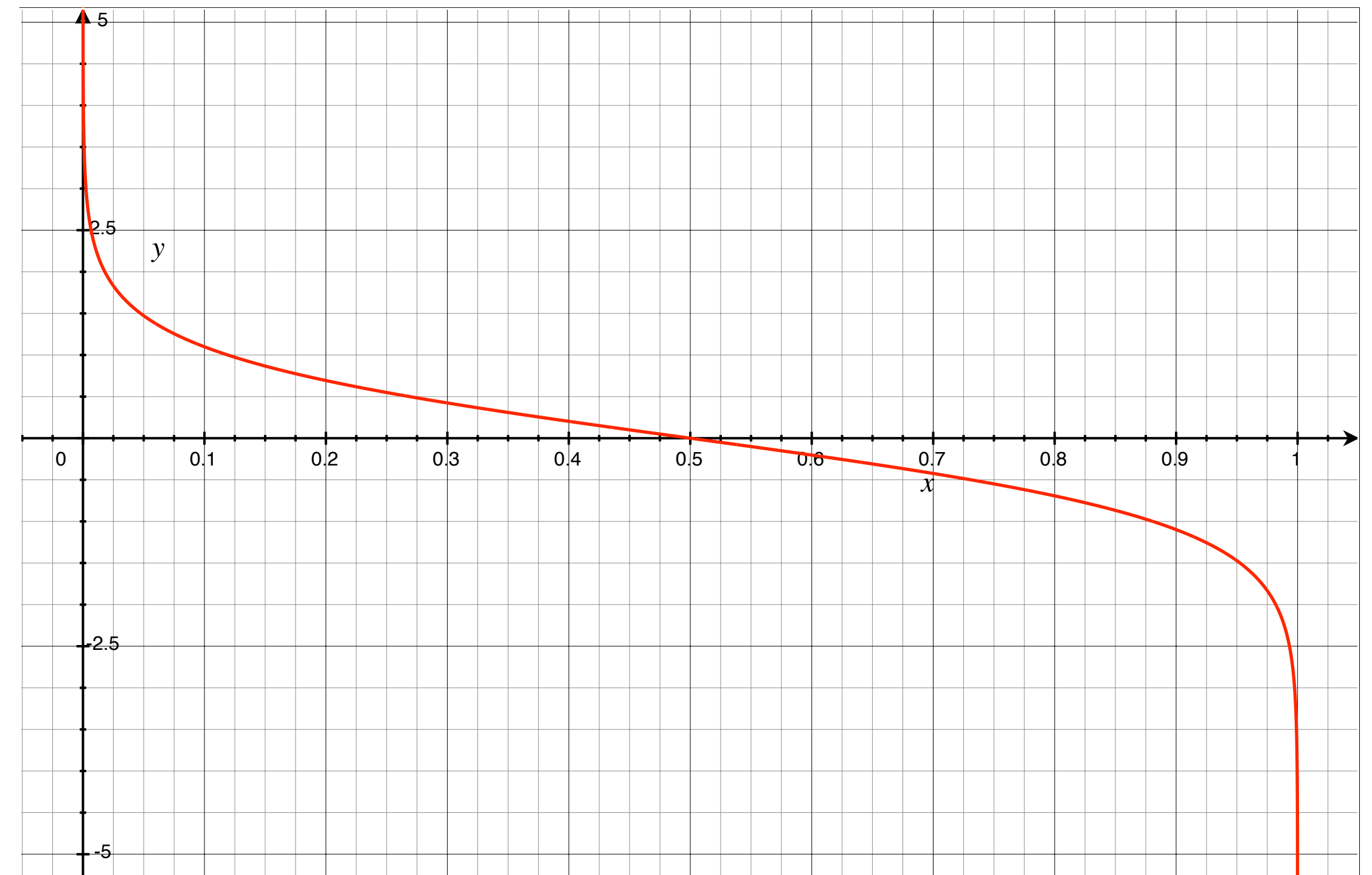


Adaptive Boosting (AdaBoost)

Agrégation

- Le modèle final M' est la moyenne pondérée des modèles séquentiels M_i
- Les poids α_i sont calculés en utilisant le taux d'erreur de chaque modèle M_i , en prenant en compte le poids de chaque observation dans le dataset D_i d'entraînement:

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \text{err}_i}{\text{err}_i} \right)$$



Gradient Boosting

Gradient Boosting (XGBoost)

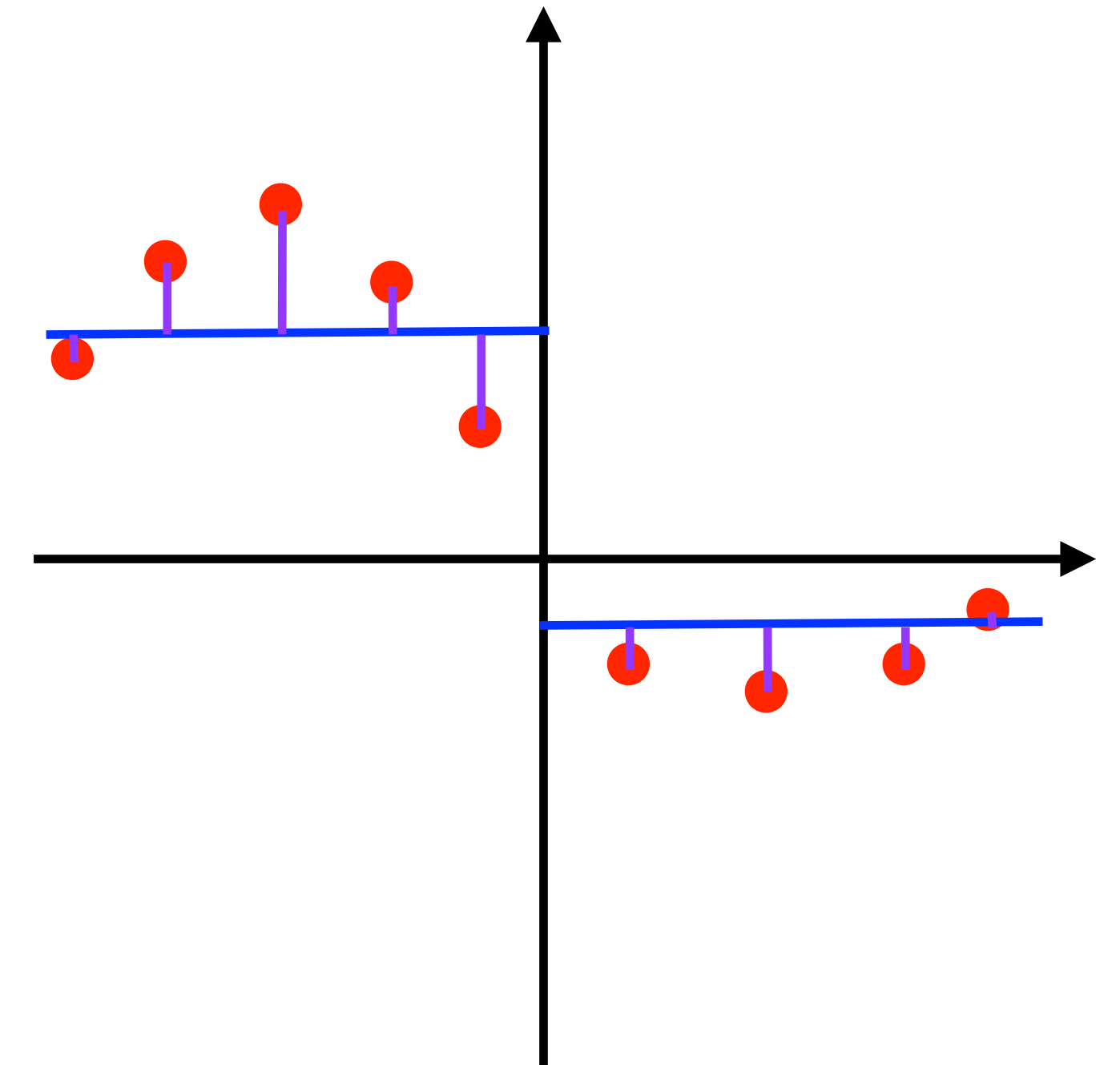
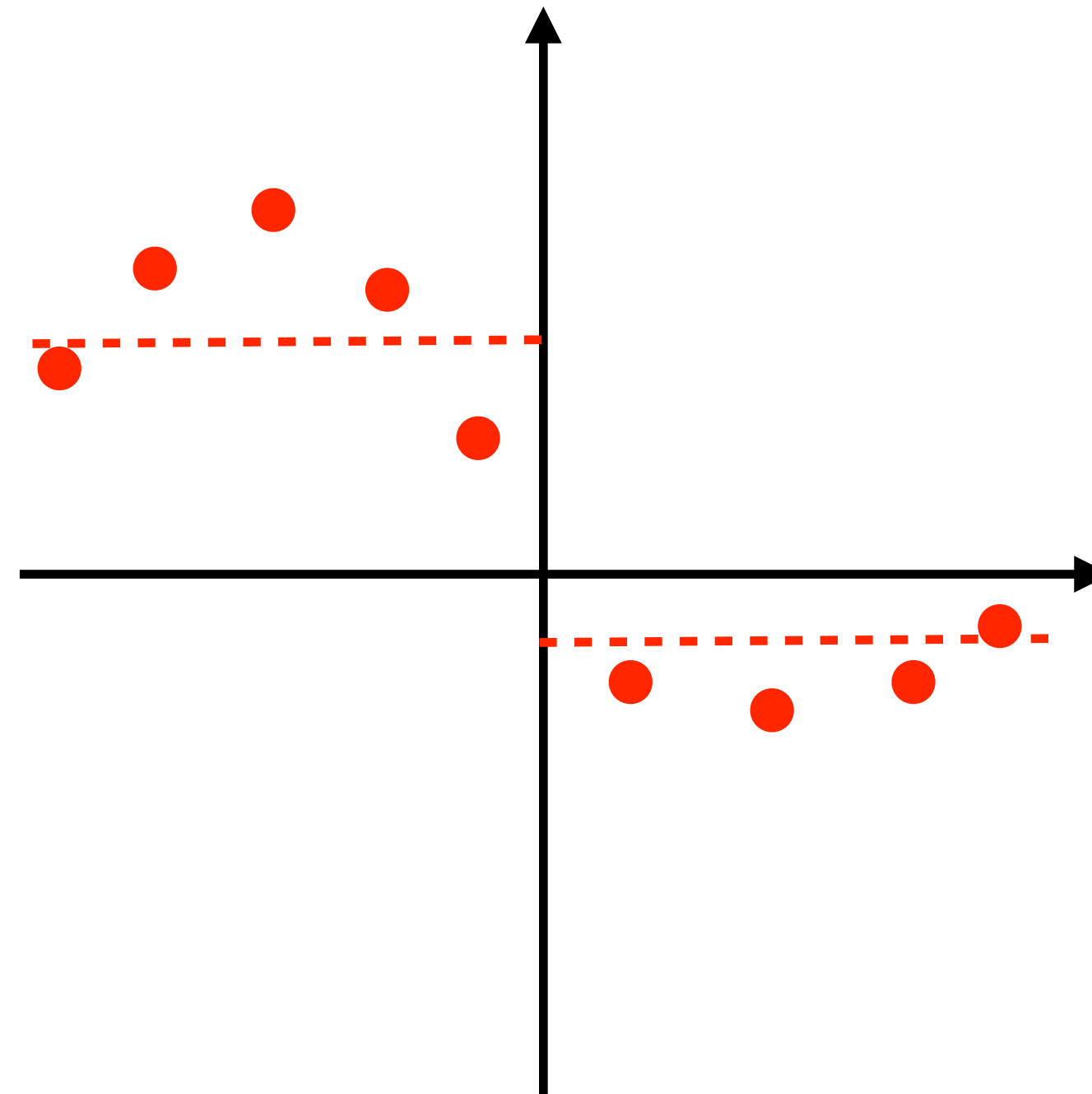
Principe

- Evolution de l'algorithme AdaBoost
- Entraînement séquentiel de modèles faibles
- Prédiction des “résidus” et non plus des données
- Modèle final basé sur un principe additif des prédictions

Gradient Boosting (XGBoost)

Exemple

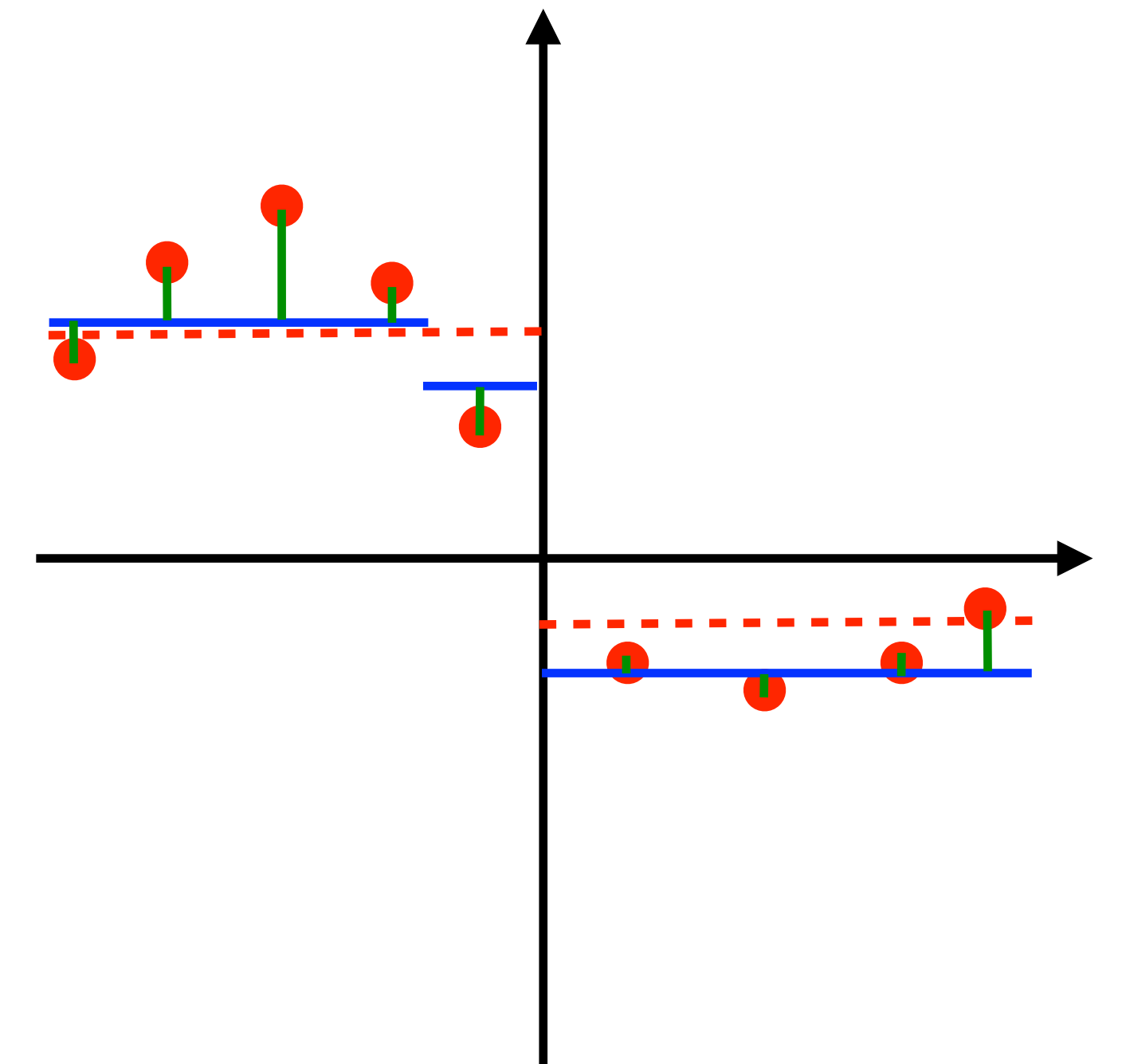
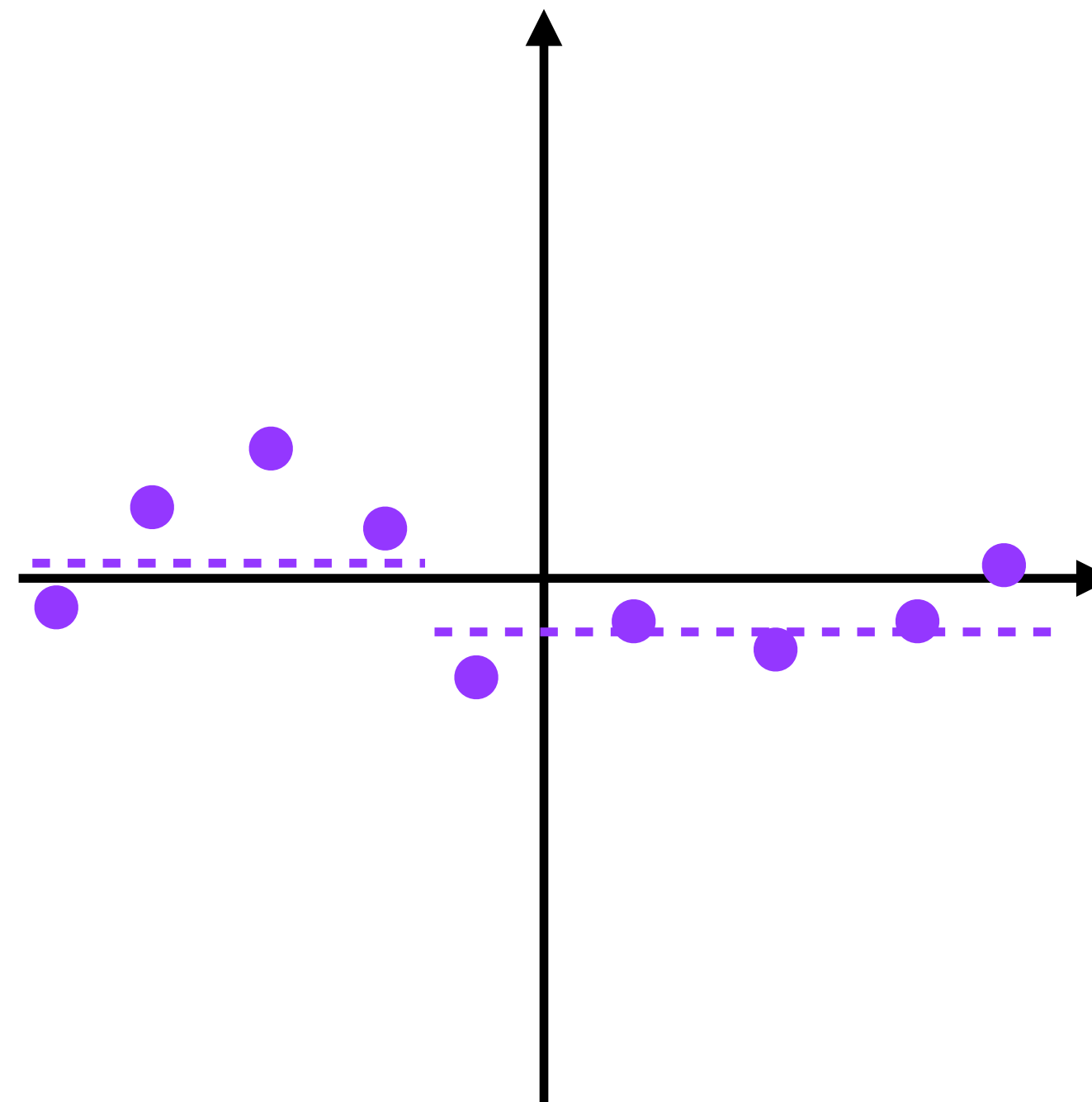
- Entraînement d'un modèle faible M_1
- Modèle $M=M_1$
- Calcul des résidus R_1



Gradient Boosting (XGBoost)

Exemple

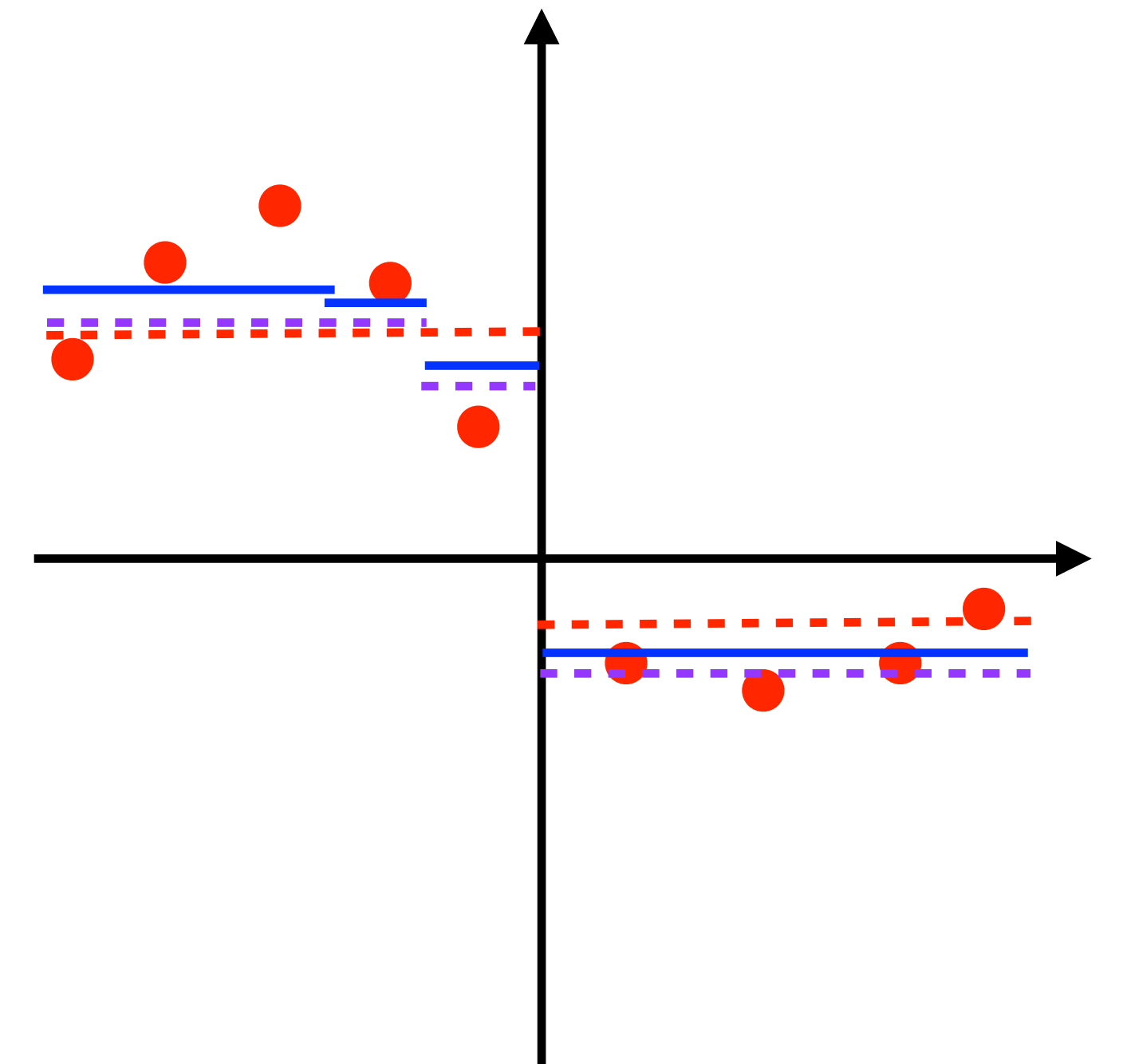
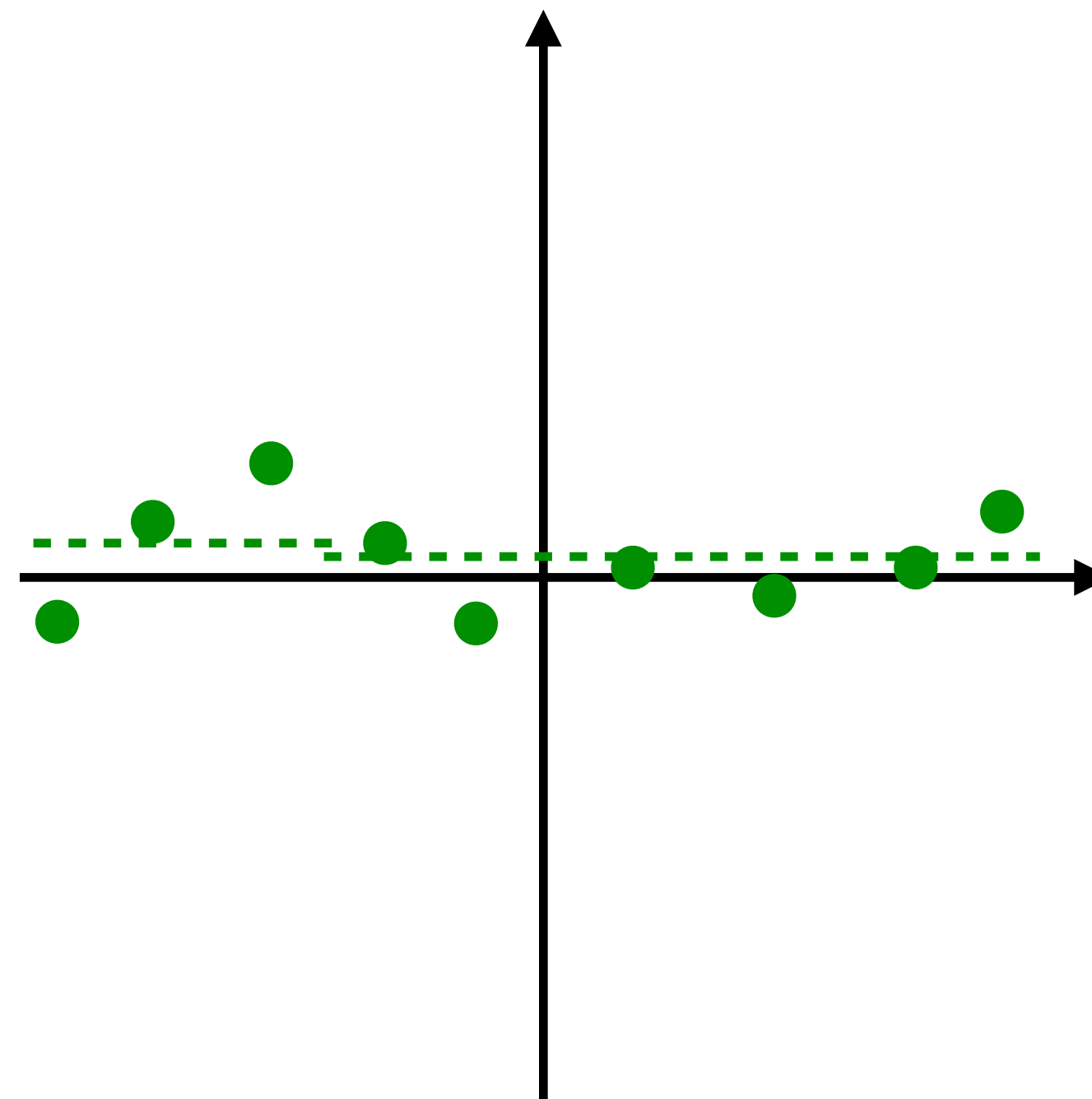
- Entraînement d'un modèle faible T_1 sur les résidus R_1
- Modèle $M = M_1 + \eta\gamma_1 T_1$
- Calcul des résidus R_2



Gradient Boosting (XGBoost)

Exemple

- Entraînement d'un modèle faible T_2 sur les résidus R_2
- Modèle $M = M_1 + \eta\gamma_1T_1 + \eta\gamma_2T_2$



Gradient Boosting (XGBoost)

Au final

- Modèle donné par:

$$M = M_1 + \eta \sum \gamma_i T_i$$

- γ_i est optimisé par descente de gradient pour estimer au mieux les données
- $\eta \in [0,1]$ est le taux d'apprentissage (learning rate)
 - Proche 1 => Apprentissage plus rapide mais risque de sur-apprentissage
- XGBoost = eXtreme Gradient Boosting
 - Optimisation algorithme du Gradient Boosting (~10 fois plus rapide)
 - Aujourd'hui une des méthode les plus utilisés dans les compétitions Kaggle

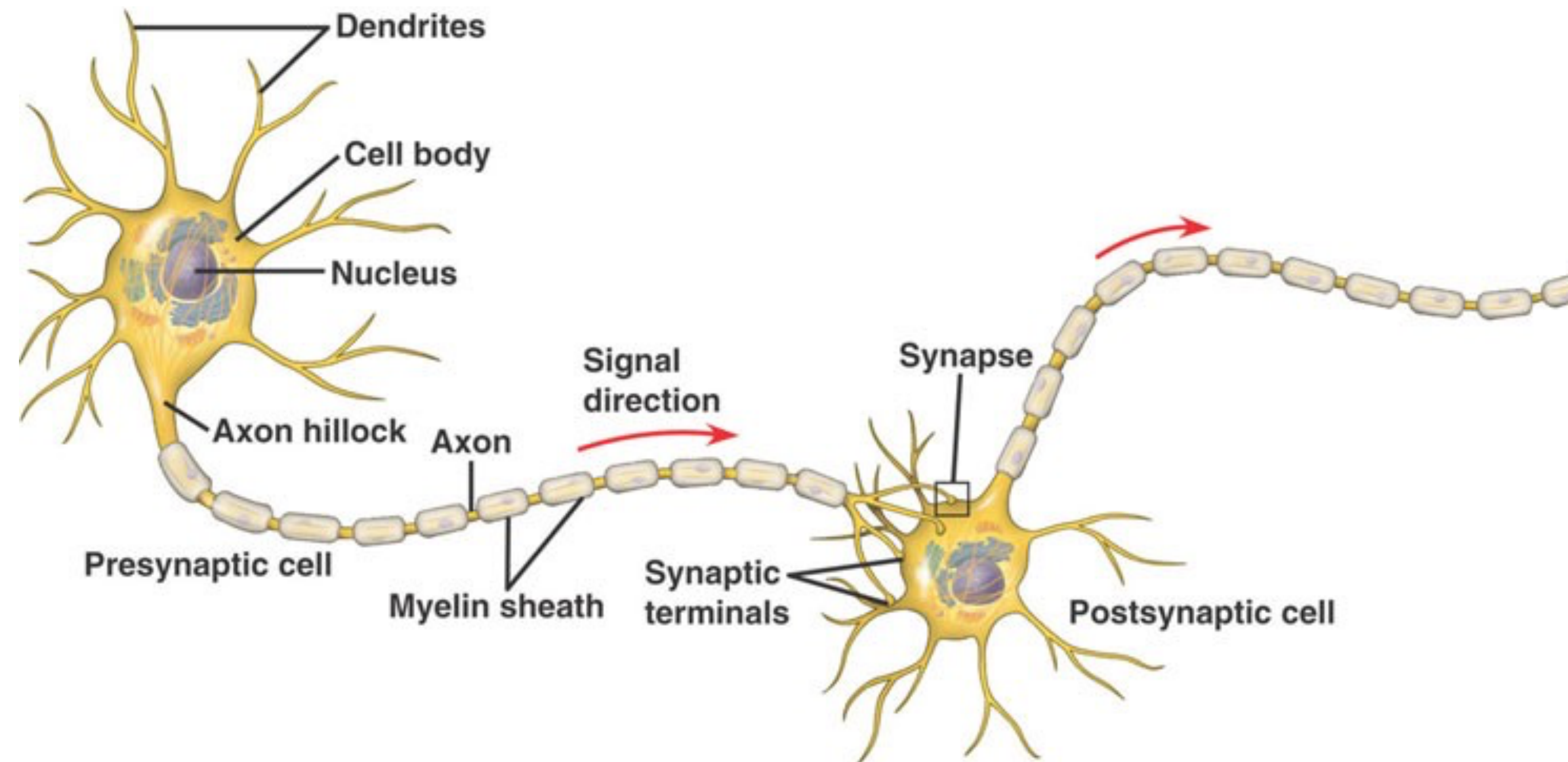
Réseaux de Neurones

Réseaux de Neurones

- Modèle mathématique (très) simplifié du fonctionnement du cerveau humain
- Combinaisons (réseau) d'éléments simples (neurones)
- Algorithme efficace pour les données non-linéaires
- Approche supervisé

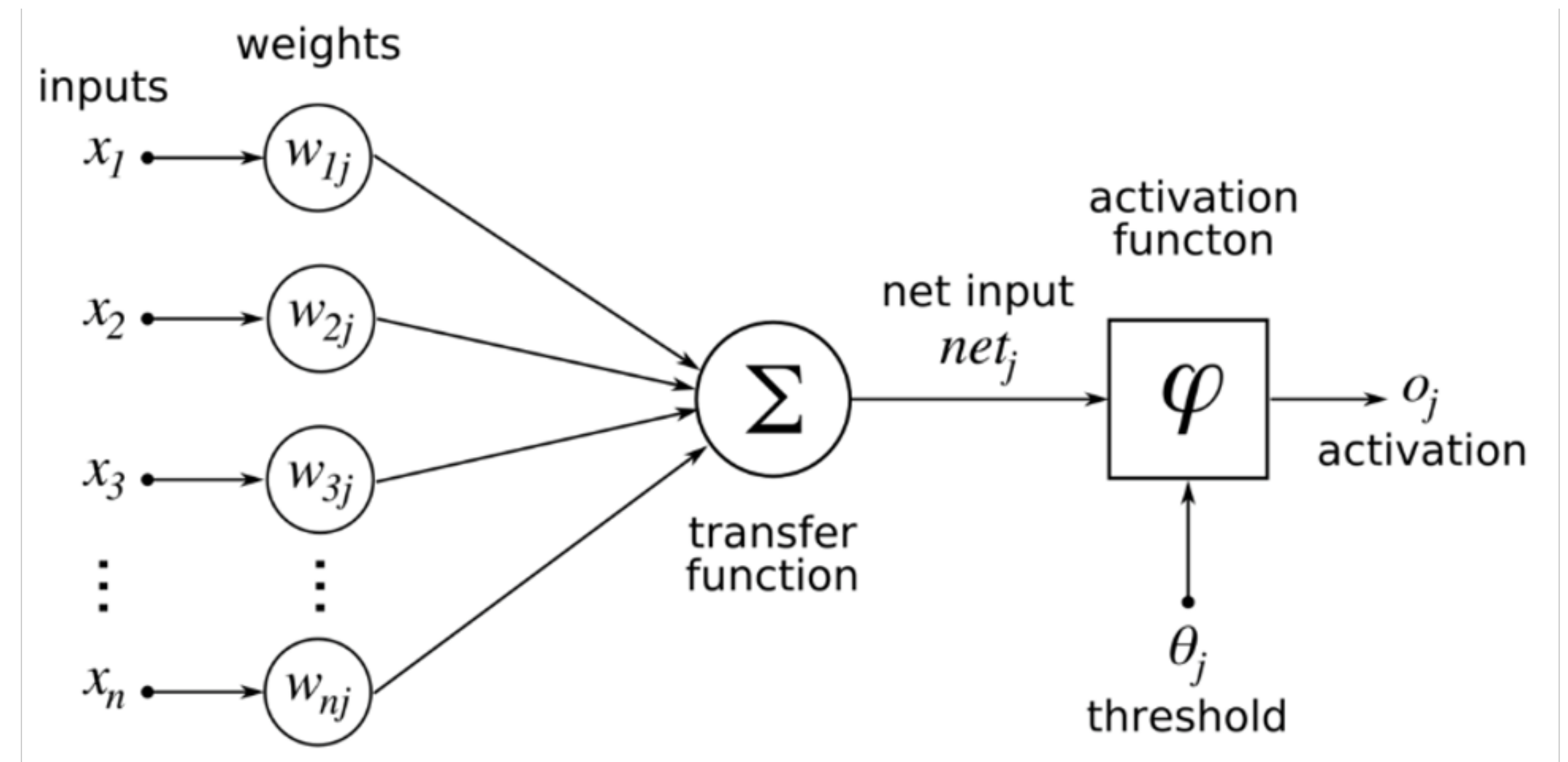
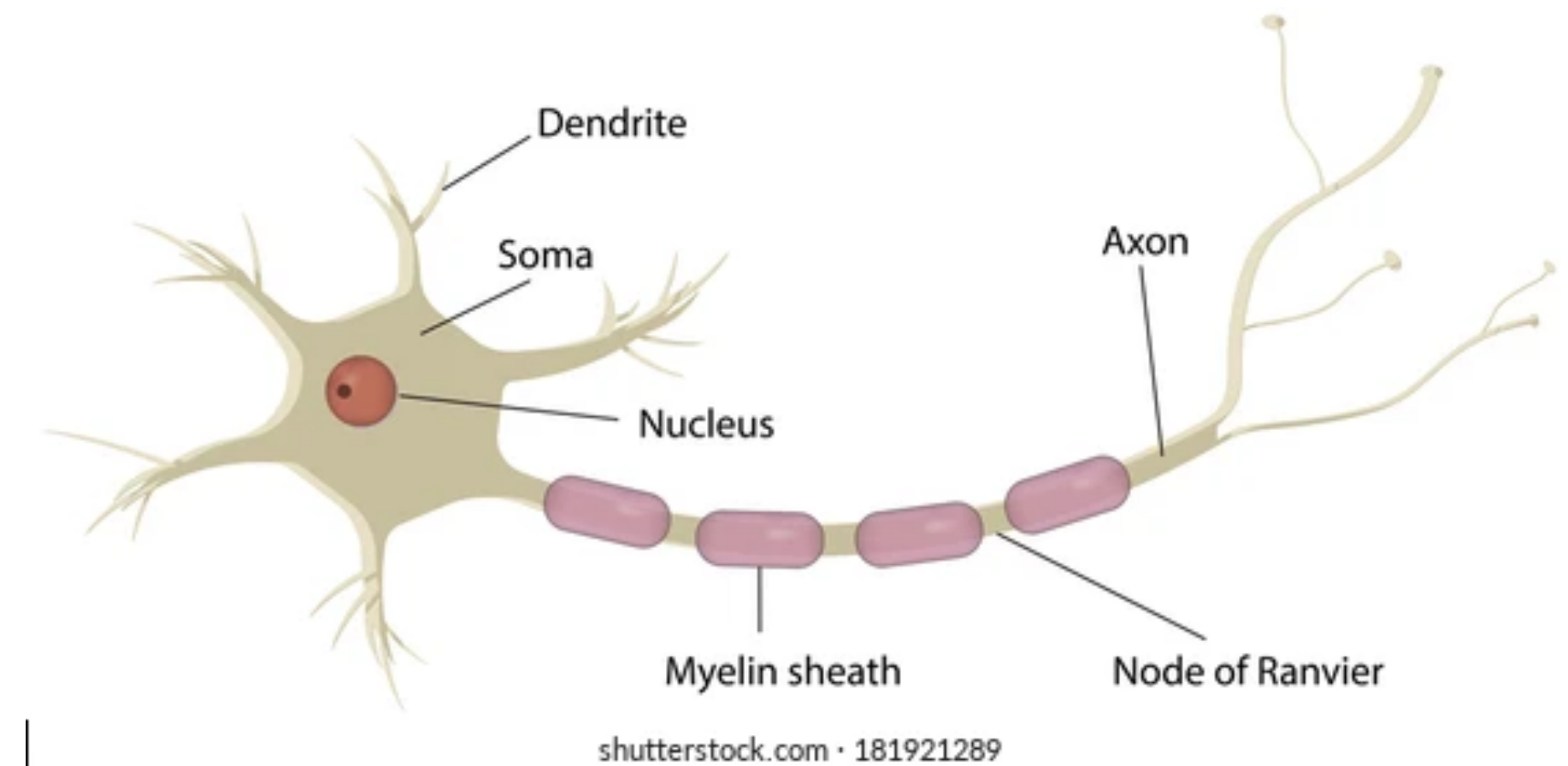
Modèle biologique

- Les **neurones** reçoivent des signaux électriques à travers leurs **dendrites**
- Les signaux sont **traités** dans le soma. Ce traitement n'est pas linéaire : effet seuil
- Le soma émet un signal électrique en fonction du traitement sur l'**axone**
- Deux neurones sont connectés (axone et dendrite) par leur **synapse**
- Cerveau humain :
 - ~100 milliards de neurones
 - ~7000 synapse par neurones => 700 trillions de connections!
 - Longueur des axones: de quelques micromètres à plusieurs mètres



Modèle mathématique

- Simplification du fonctionnement d'un neurone
 - Pas d'aspect temporel
 - Somme pondérée (poids synaptique) des signaux entrants
 - Sortie fournie par une fonction d'activation



Modèle mathématique

- Construire un réseau de neurones consiste à :
 - Trouver l'architecture du réseau
 - Optimiser les poids synaptiques

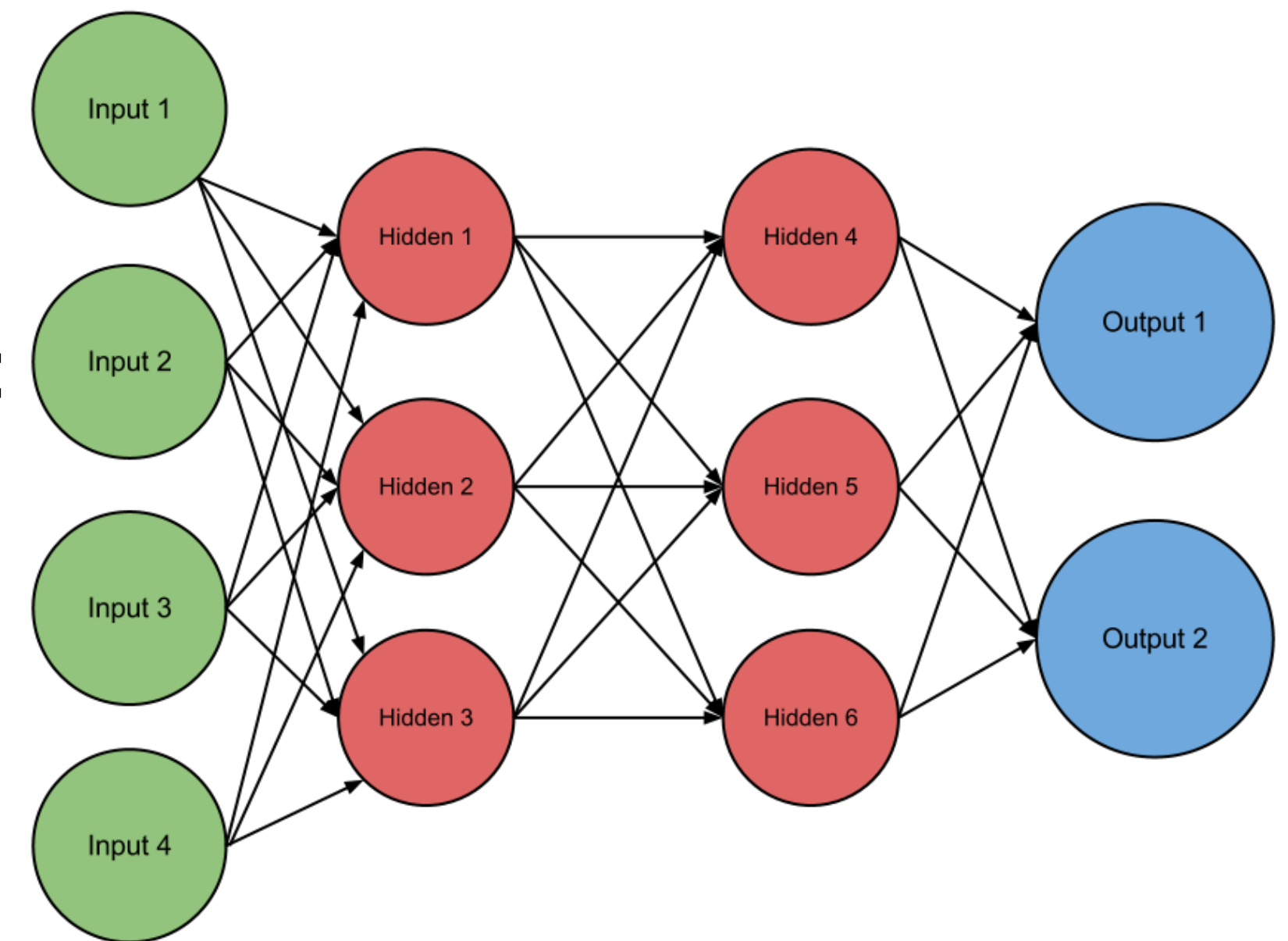


Perceptron Multi-Couche

MultiLayer Perceptron (MLP)

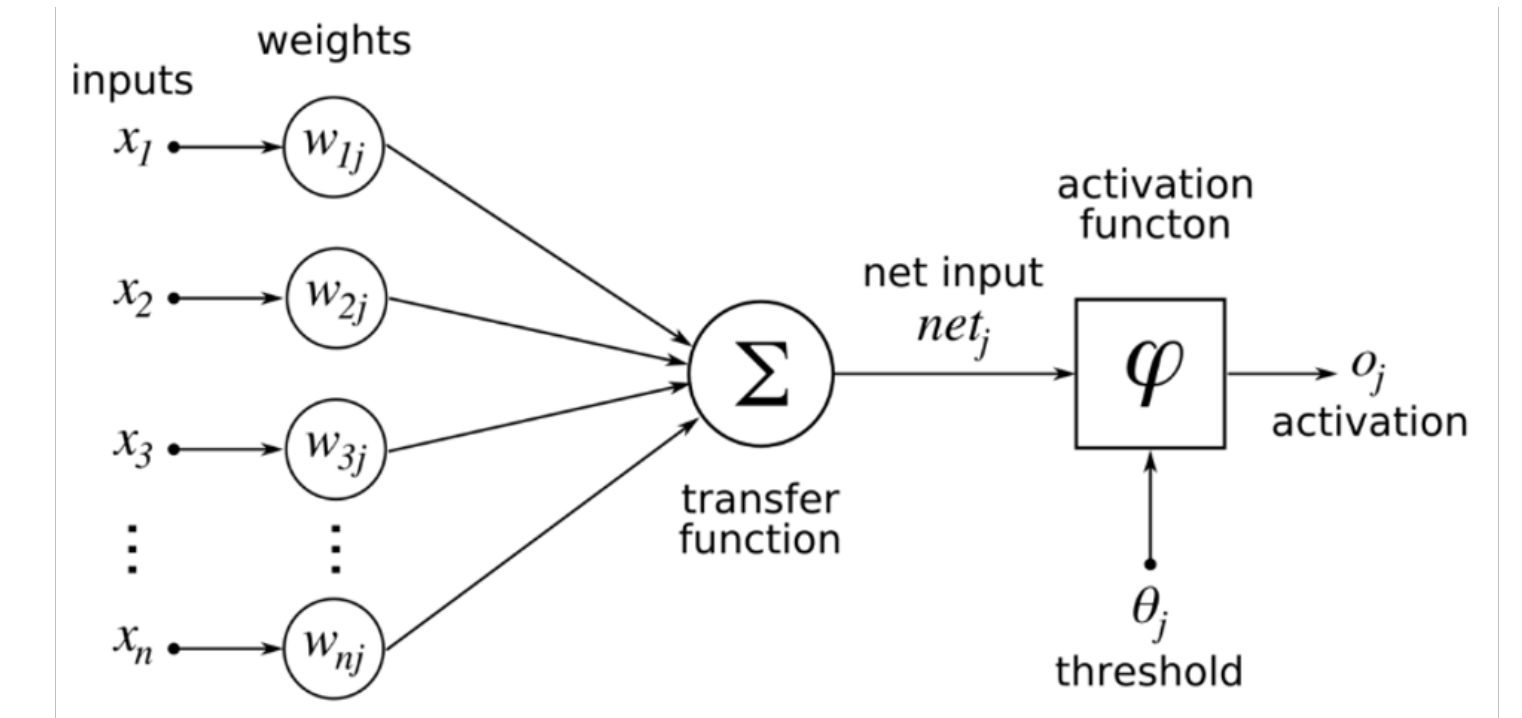
Principe

- Architecture spécifique de réseaux de neurones relativement simple à utiliser
- Couches cachées :
 - 1 couche : Perceptron => Modèle linéaire des données
 - >1 couches : Multi-Layer Perceptron
- Basés sur les réseaux de neurones à **propagation avant** :
Les signaux transitent toujours des entrées vers les sorties (pas de retour en arrière possible dans le réseau)
- Apprentissage souvent basé sur la **rétropropagation du gradient** (backpropagation)



MultiLayer Perceptron (MLP)

Fonctions d'activation

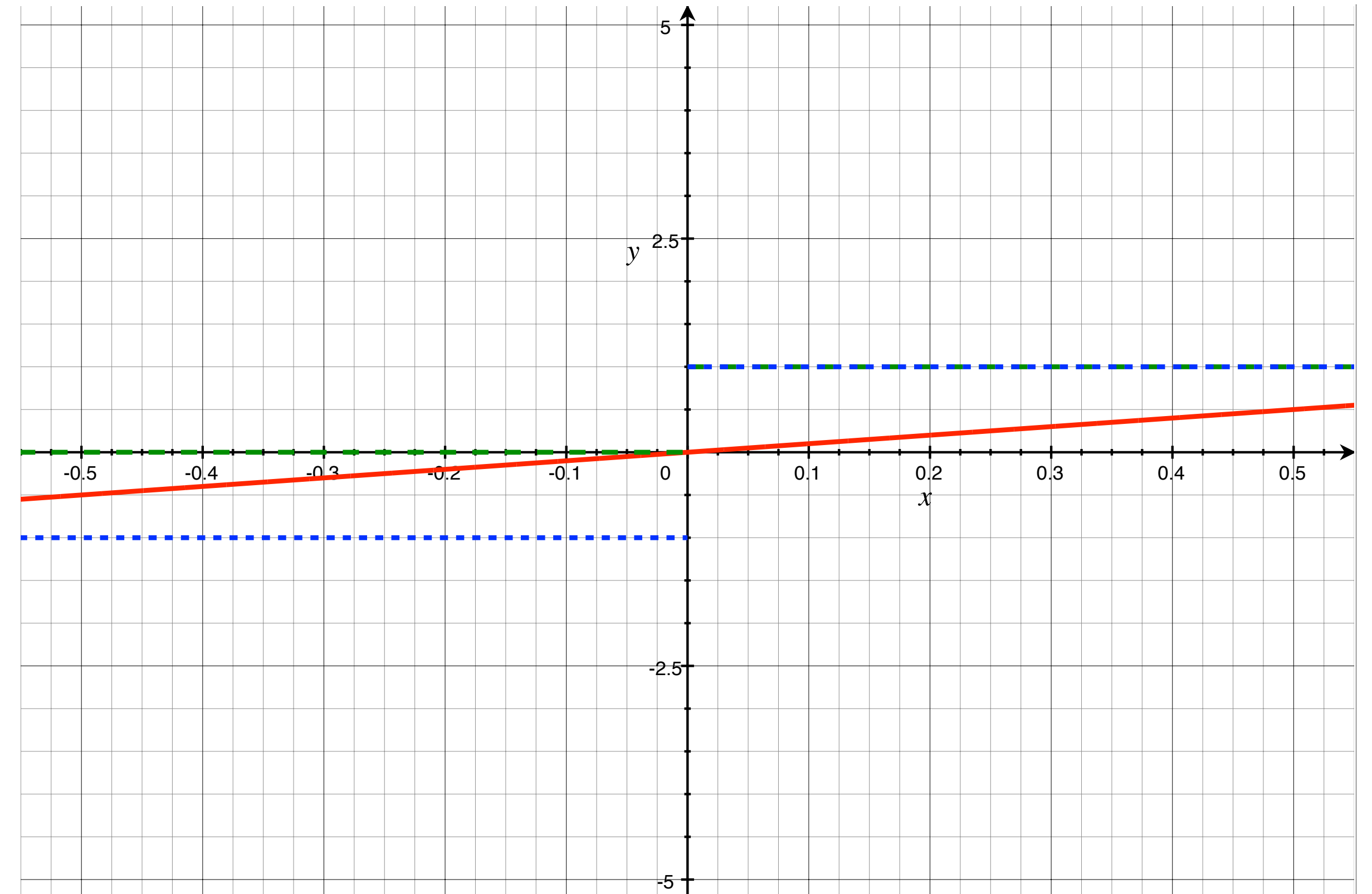


- Utilisation de fonctions d'activation non linéaire

- Linéaire : $\varphi(x) = x$

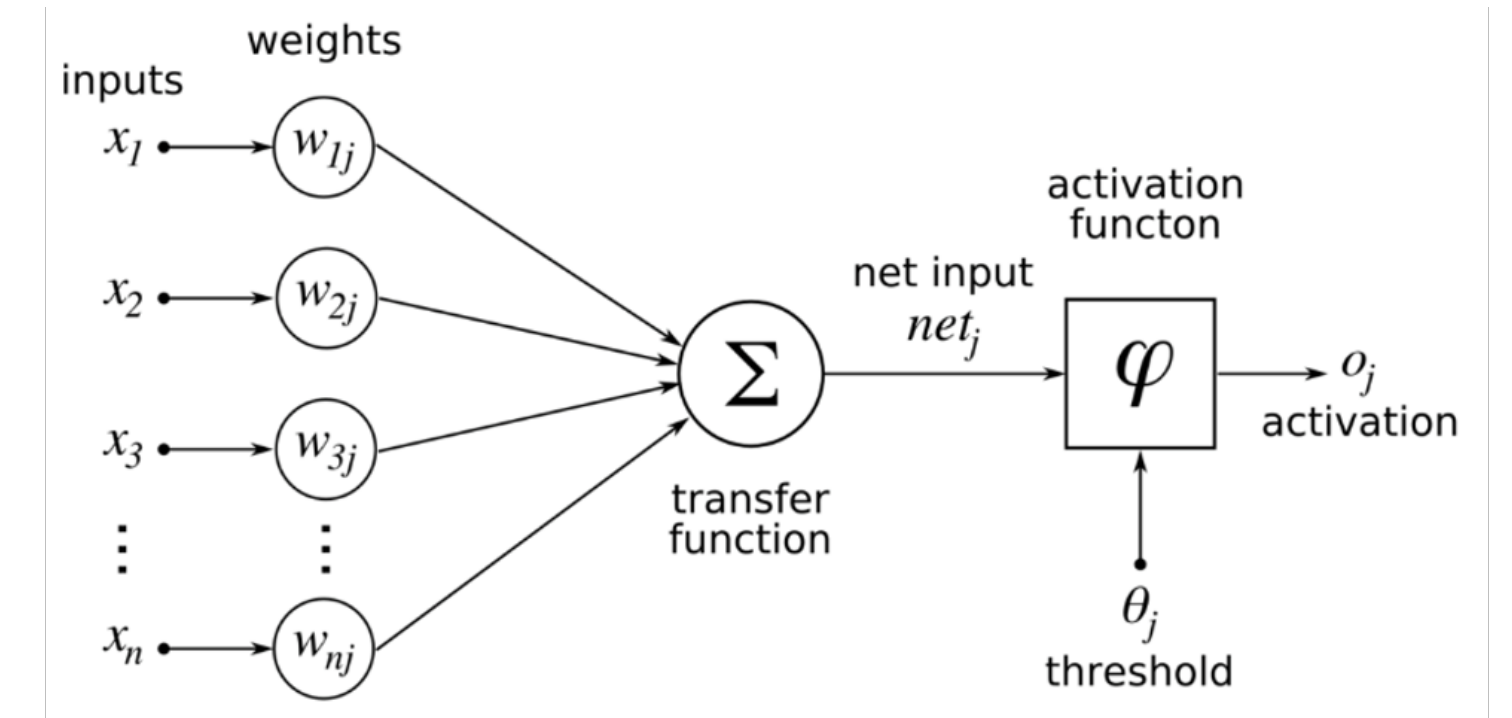
- Heavyside : $\varphi(x) = \begin{cases} 0 & \text{if } x < 0 \\ 0.5 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases}$

- Signe : $\varphi(x) = \begin{cases} -1 & \text{if } x < 0 \\ 0 & \text{if } x = 0 \\ 1 & \text{if } x > 0 \end{cases}$



MultiLayer Perceptron (MLP)

Fonctions d'activation



- Utilisation de fonctions d'activation non linéaire

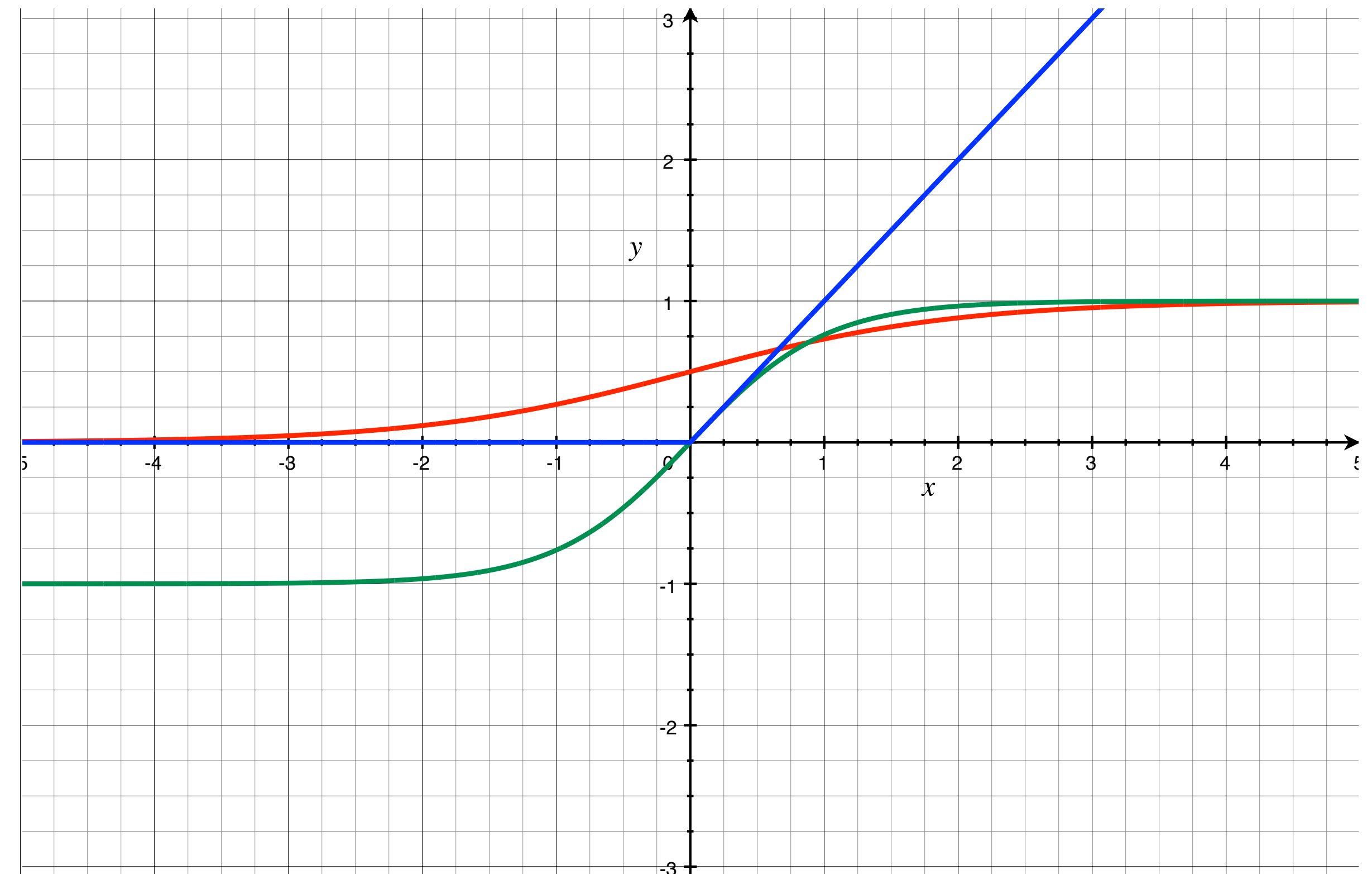
- Sigmoid : $\varphi(x) = \frac{1}{1 + e^{-\beta x}}$

- Arctan (hypertangent) : $\varphi(x) = \tanh(x)$

- Rectified Linear Unit (ReLU) :

$$\varphi(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{otherwise} \end{cases}$$

- Softmax : $\varphi(x_i) = \frac{e^{x_i}}{\sum_{j=1}^M e^{x_j}}$



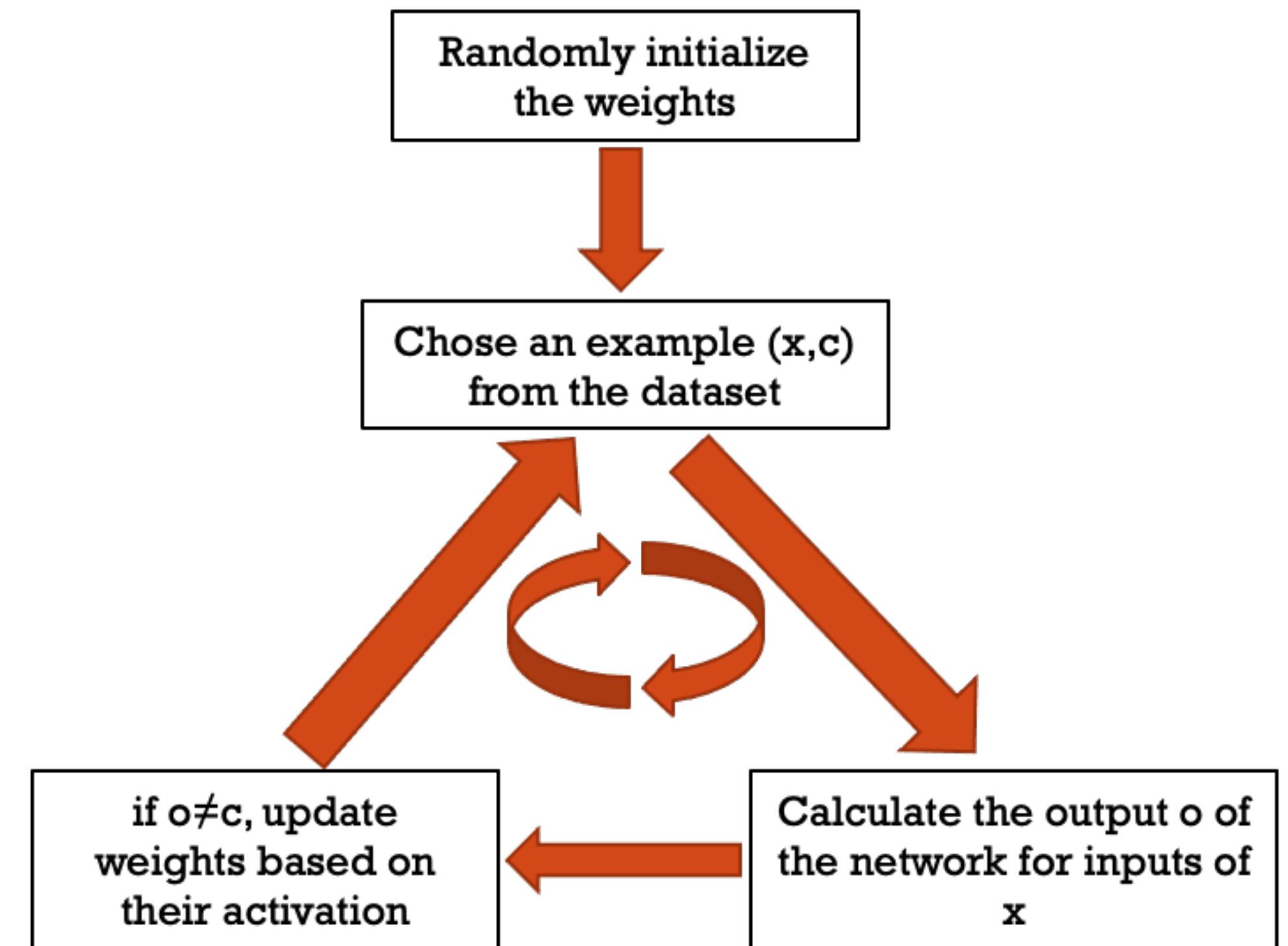
Loi de Hebb (1949) - Apprentissage

- Principe : Si 2 neurones sont activés en même temps alors leur connexion se renforce
- Objectif :
Soit S un dataset d'entraînement composés de couple (x, c) :
 - x est un vecteur d'entrées $(x_1, x_2, \dots, x_n)$
 - c est un vecteur de sorties attendues $(c_1, c_2, \dots, c_n)$On cherche l'ensemble des poids synaptiques w_i qui minimisent les différences entre la sortie o du réseau et la sortie attendue c pour tout le dataset d'entraînement

Loi de Hebb (1949) - Apprentissage

Algorithme

```
Randomly initialize the weights  $w_i$ 
while TRUE
- Take a example  $(x, c)$  from  $S$ 
- Calculate the output  $o$  of the
  network for the input  $x$ 
- Update the weights of the
  network as follows:
  for  $i$  in  $[1, n]$ :
     $w_i = w_i + \varepsilon * (c - o) * x_i$ 
  end for
end while
```



Loi de Hebb (1949) - Apprentissage

Exemple

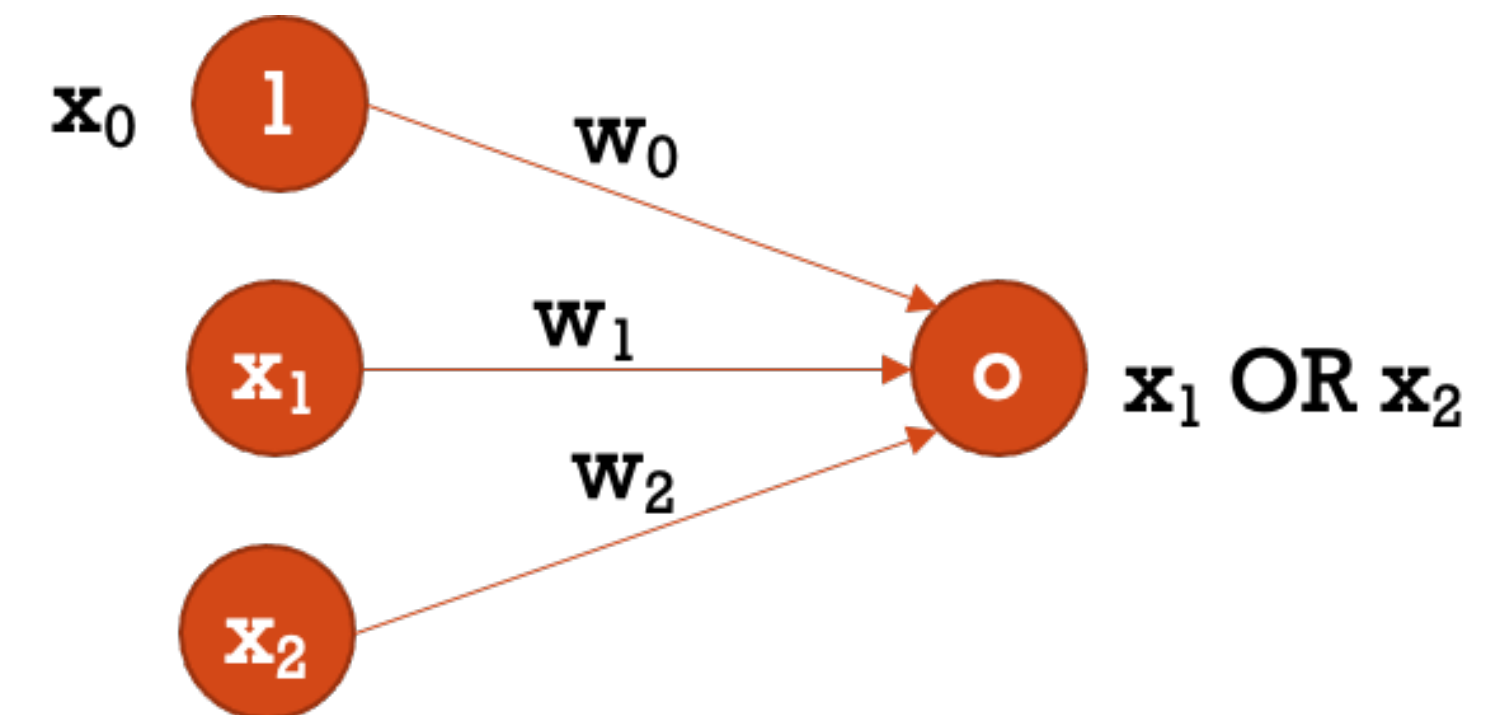
- On cherche à entraîner un réseau de neurones reproduisant la fonction logique OR

- 2 entrées x_1 et x_2 et 1 biais
- 1 sortie attendue $c = x_1 \text{ OR } x_2$

- Fonction d'activation :

$$\varphi(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$

- Learning rate : $\varepsilon = 1$



x_1	x_2	c
0	0	0
0	1	1
1	0	1
1	1	1

Loi de Hebb (1949) - Apprentissage

Exemple

Step t	w ₀ (t)	w ₁ (t)	w ₂ (t)	x ₀	x ₁	x ₂	$\sum w_i x_i$	o	c	w ₀ (t+1)	w ₁ (t+1)	w ₂ (t+1)
Init										0	1	-1
1				1	0	0			0			
2				1	0	1			1			
3				1	1	0			1			
4				1	1	1			1			
5				1	0	0			0			
6				1	0	1			1			
7				1	1	0			1			
8				1	1	1			1			
9				1	0	0			0			
10				1	0	1			1			

Loi de Hebb (1949) - Apprentissage

Exemple - Solution

Step t	w ₀ (t)	w ₁ (t)	w ₂ (t)	x ₀	x ₁	x ₂	∑w _i x _i	o	c	w ₀ (t+1)	w ₁ (t+1)	w ₂ (t+1)
Init										0	1	-1
1	0	1	-1	1	0	0	0	0	0	0+0*1*ε	1+0*0*ε	-1+0*0*ε
2	0	1	-1	1	0	1	-1	0	1	0+1*1*ε	1+1*0*ε	-1+1*1*ε
3	1	1	0	1	1	0	2	1	1	1	1	0
4	1	1	0	1	1	1	2	1	1	1	1	0
5	1	1	0	1	0	0	1	1	0	1+(-1)*1*ε	1+(-1)*0*ε	0+(-1)*0*ε
6	0	1	0	1	0	1	0	0	1	0+1*1*ε	1+1*0*ε	0+1*1*ε
7	1	1	1	1	1	0	2	1	1	1	1	1
8	1	1	1	1	1	1	3	1	1	1	1	1
9	1	1	1	1	0	0	1	1	0	1+(-1)*1*ε	1+(-1)*0*ε	1+(-1)*0*ε
10	0	1	1	1	0	1	1	1	1	0	1	1

Apprentissage dans la pratique

- Le choix du learning rate est central
 - Trop gros : Risque d'oscillation autour de l'erreur minimum
 - Trop petit : Nombre d'itérations important avant convergence
- A l'heure actuelle
 - Utilisation de la descente de gradient (SGD) pour moduler l'apprentissage
$$w_i = w_i + \varepsilon * (c - o) * x_i * \sigma'(x.w)$$
 - Et même méthode adaptative avec l'optimiseur Adam (Adaptive moment estimation) qui auto-paramètre la descente de gradient

Apprentissage dans la pratique

Paramètres de l'optimiser Adam

- alpha : learning rate.
Les grandes valeurs (>0.1) permettent une convergence plus rapide en début d'entraînement
Les petites valeurs ($\sim 10^{-5}$) ralentissent l'apprentissage durant l'entraînement
- beta1 : taux de décroissance exponentielle pour le premier moment d'estimation. Par défaut 0.9 (rarement modifié)
- beta2. taux de décroissance exponentielle pour le second moment d'estimation. Par défaut 0.999 (rarement modifié)

LBFGS : Autre algorithme d'optimisation

- Méthode quasi-Newtonienne d'optimisation par descente de gradient
- Utilisation d'une matrice de gradients pour guider l'algorithme
- Méthode très efficace pour trouver un minimum local mais couteuse en temps de calcul
- Utilisé pour les petits datasets

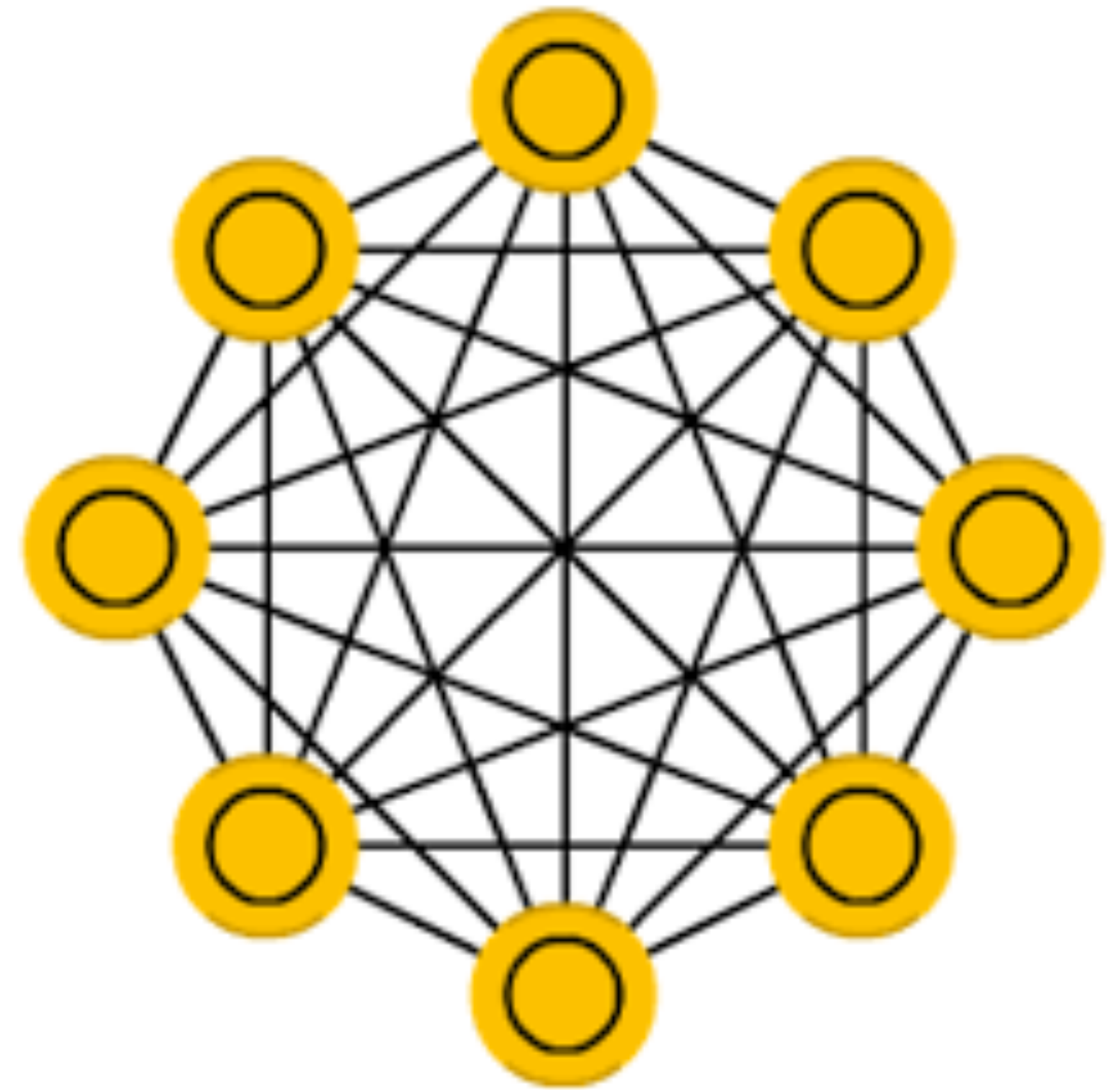
Comment trouver l'architecture du perceptron?

- Apprentissage relativement simple à gérer
- Problème central du perceptron: Nombre et taille des couches cachées
- Pas de méthodologie claire à ce sujet mais
 - Procéder par essai erreur : commencer par des réseaux peu profond et augmenter la profondeur et la largeur progressivement
 - Intuition à avoir par rapport à l'analyse des données
 - Données simples et relativement corrélées => réseau simple
 - Données peu corrélées => réseau plus profond
 - Rechercher des architectures existantes sur des problèmes similaires
 - Recherche systématique (GridSearch par exemple)

Autres architectures

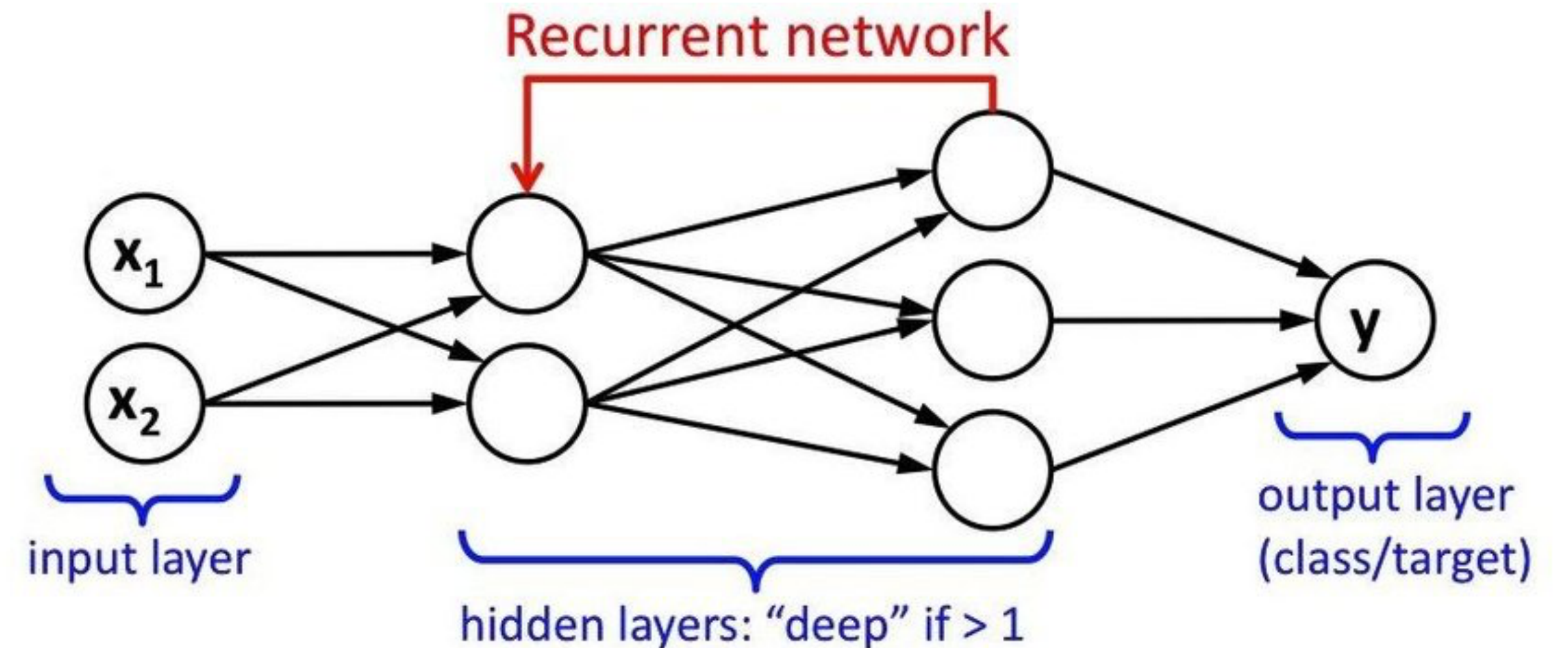
Réseaux de Hopfield

- Réseaux complètement connectés
- Permet de créer de la mémoire en interne
- Apprentissage très complexe et couteux en temps de calcul



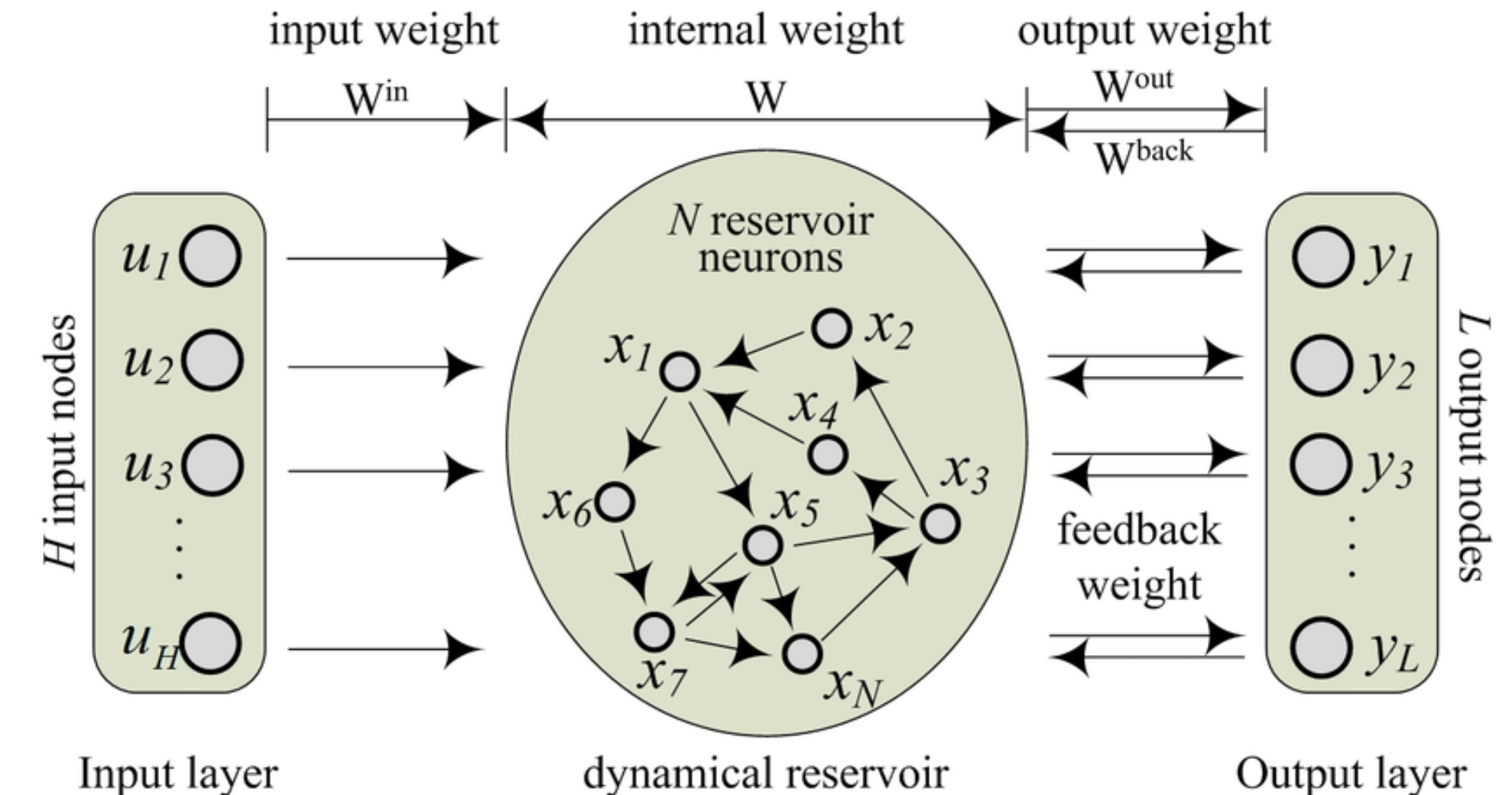
Recurrent Neural Networks (RNN)

- Permet de prendre en compte la temporalité dans les données (séries temporelles par exemple)
- Capacité de mémorisation dans le réseau



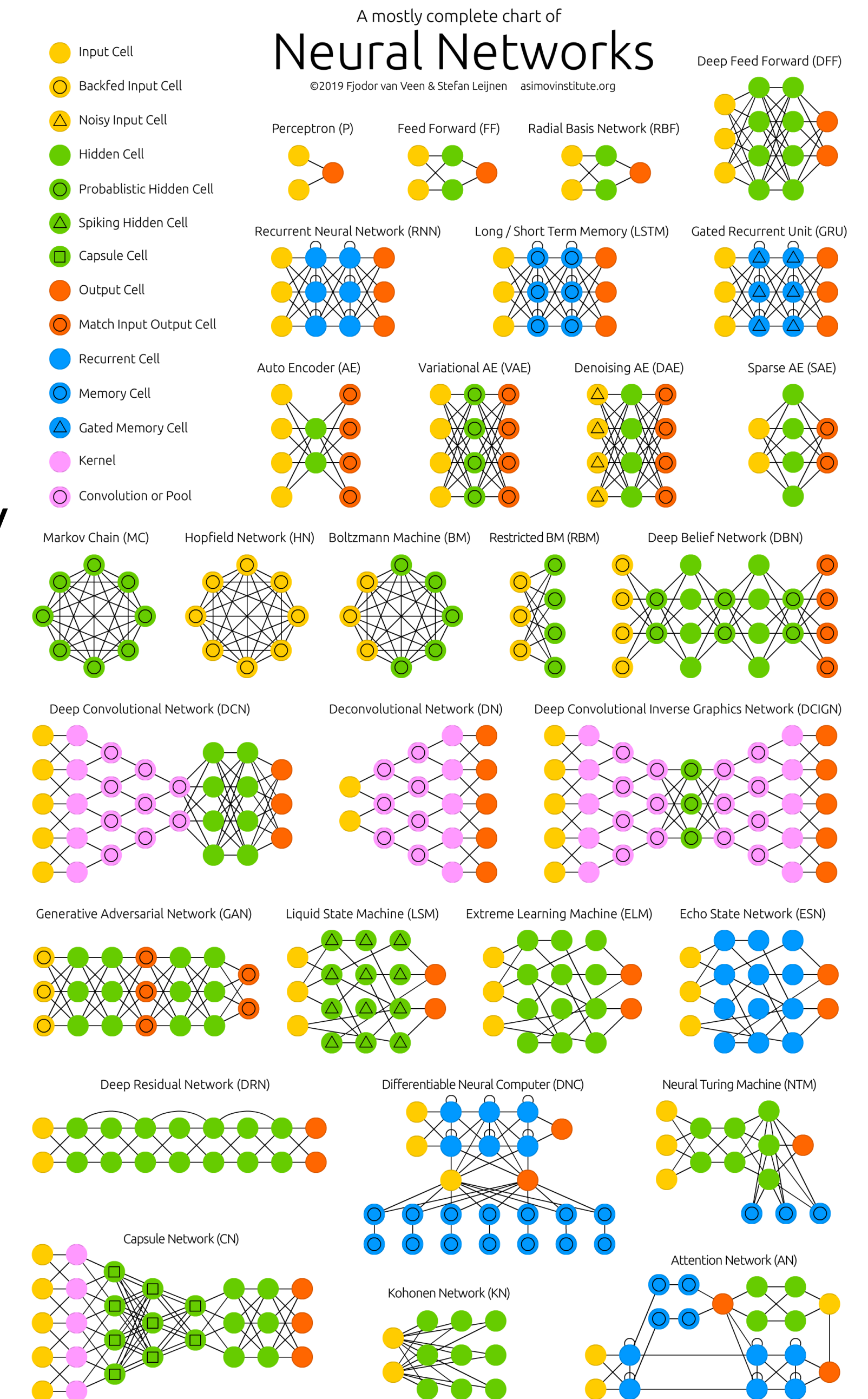
Echo State Networks (ESN)

- Simplification et version plus efficace des RNN
- Réseaux récurrents peu connectés (~1% de connectivité dans les couches cachées)
- Poids internes aléatoires, seuls les poids de la couche de sortie sont optimisés



Et bien d'autres...

- Neural Network Zoo :
<https://www.asimovinstitute.org/neural-network-zoo/>
- Voir le cours de Deep Learning en M2



Feature selection

Contexte

- Problèmes : dataset avec des centaines, voire milliers de features.
 - Problème extrême : "small n large p" : plus de features que de sample (données) (e.g. données biomédicales).
 - Risque important d'**over-fitting**.
- Mais toutes les features sont-elles intéressantes pour construire le modèle ?
 - Feature selection = méthodes pour ne sélectionner que les features importantes.

Avantages

- **Feature selection** permet de :
 - Accélérer l'apprentissage des algorithmes de ML
 - Réduire la complexité du modèle (voire le rendre plus facile à interpréter)
 - Améliorer la qualité du modèle (si le dataset est bien choisi).
 - Réduire l'overfitting.

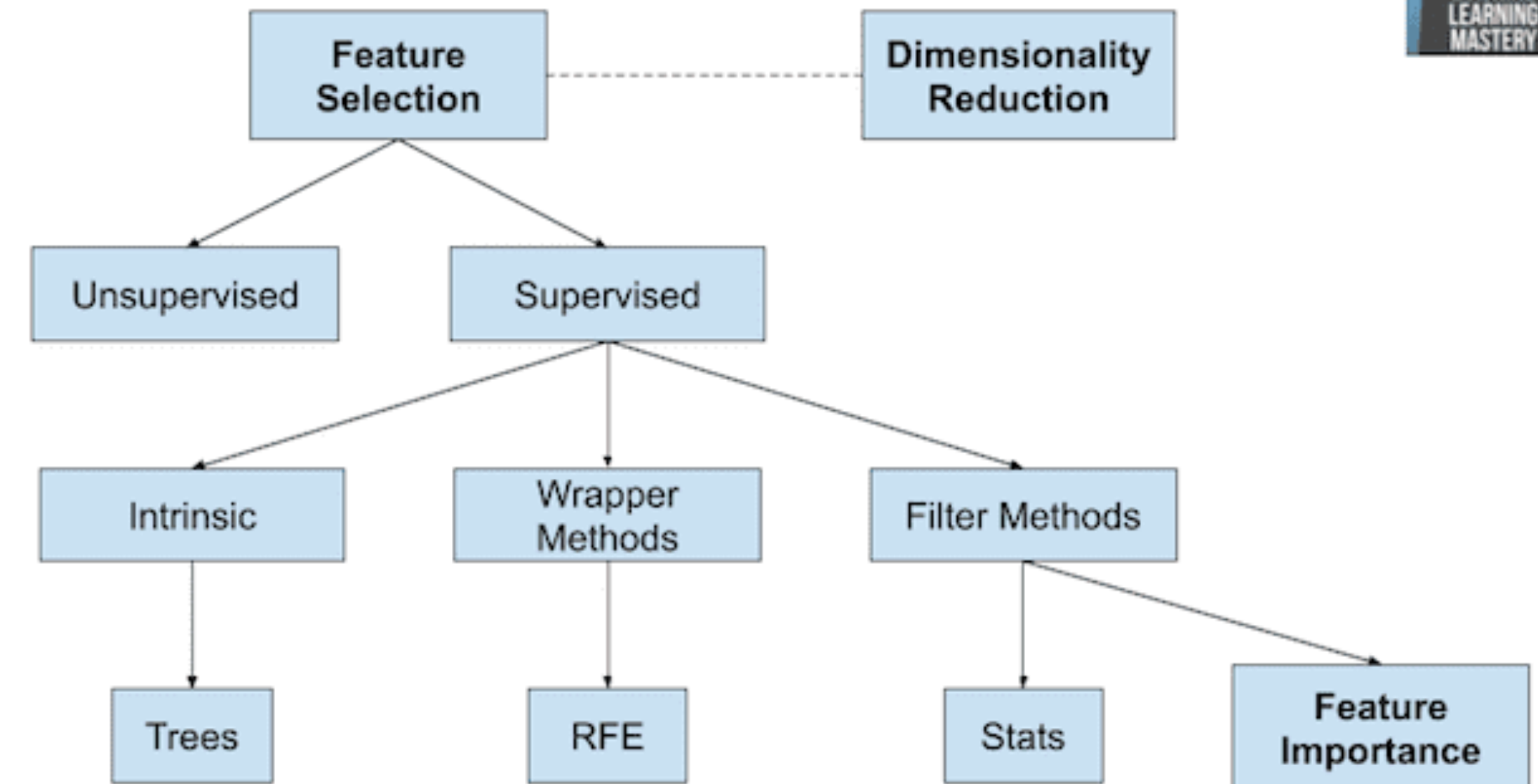
Feature selection vs Réduction de dimensionnalité

- **Ces deux approches cherchent à réduire le nombre de variables d'entrées dans les algorithmes de ML.**
- **Réduction de dimensionnalité** crée une projection dans un nouvel espace
➡ proposition d'un ensemble de nouvelles features.
- **Feature selection** choisit certaines features du dataset.
➡ Sélection des features parmi les features du dataset

Panorama

- **Non-Supervisé** : N'utilise pas la variable cible, mais tente de supprimer des variables redondantes => **Correlation**
- **Supervisé** : Utilisation de la variable cible
 - **Intrinsèque** : Les algorithmes font automatiquement la sélection pendant l'entraînement => ex: Decision Trees
 - **Filtre** : Choisissent des sous-ensembles de features en se basant sur leur relation avec la cible => **méthodes statistiques**
 - **Wrapper** : Recherche de « bons » sous-ensembles de features => **Boruta**

Overview of Feature Selection Techniques

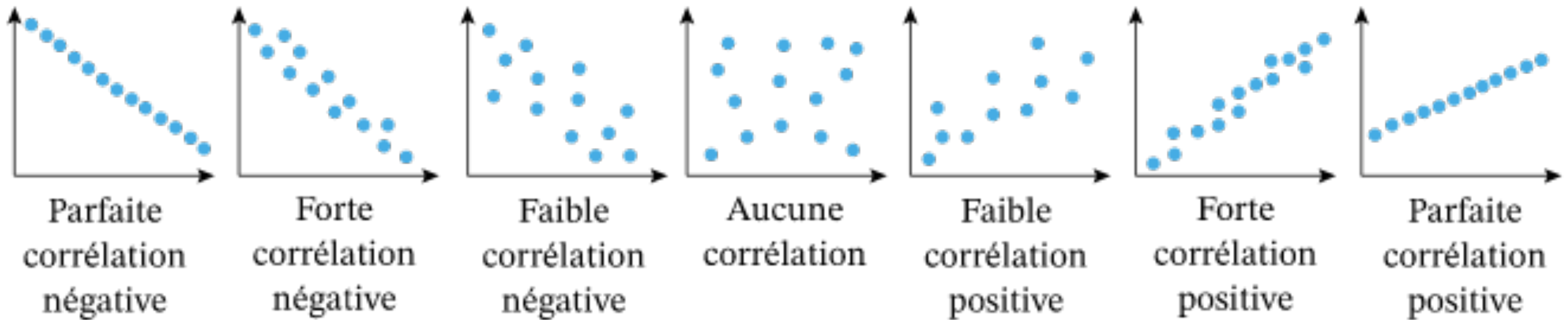


Copyright © MachineLearningMastery.com

<https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/>



Correlation (de Pearson)

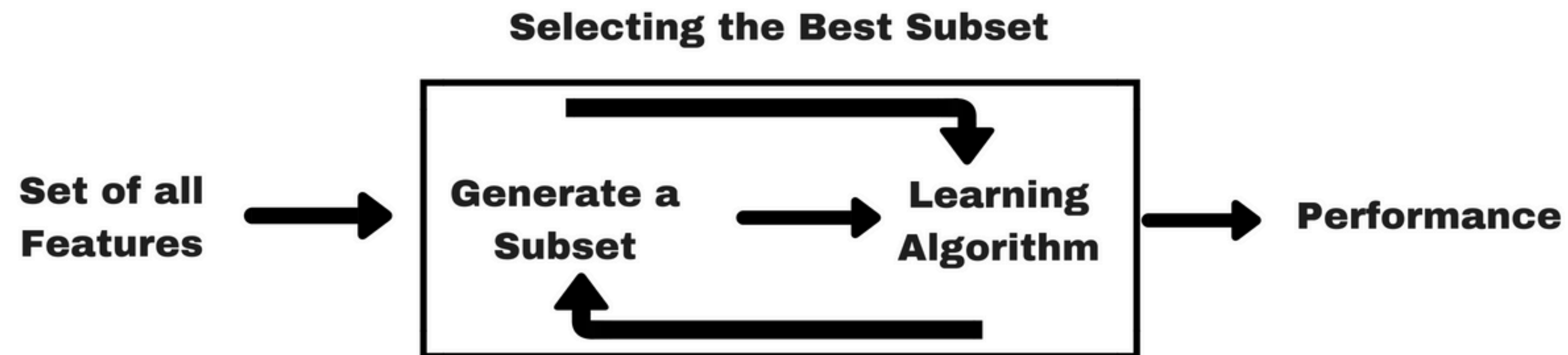


Panorama

- Méthodes par filtrage



- Méthodes par wrapper



Feature selection

Méthode par filtrage

Méthodes par filtrage

- Principe : utiliser des indicateurs statistiques (corrélation...) pour déterminer les features intéressantes.
- Approches utilisées en pré-processing
- Indépendant de tout algorithme de ML.
- En général, méthodes univariées, peu efficaces sur des datasets réels (relations cachées entre les features). Méthodes multivariées : choix de features optimisant un indicateur.



Méthodes par filtrage

- La méthode dépend du type des features et données de sortie

Feature\Response	Continuous	Categorical
Continuous	Pearson's Correlation	LDA
Categorical	Anova	Chi-Square

Méthodes par filtrage

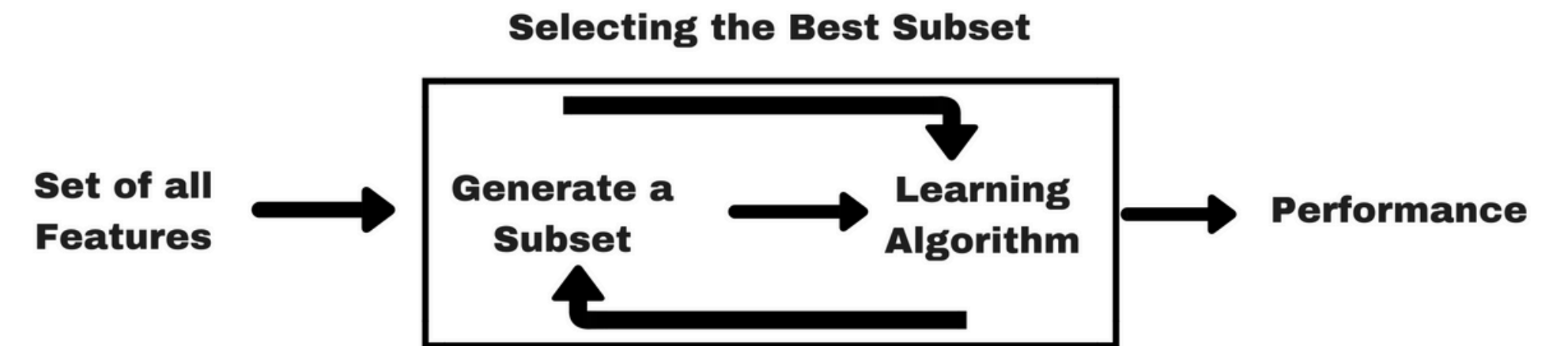
Feature\Response	Continuous	Categorical
Continuous	Pearson's Correlation	LDA
Categorical	Anova	Chi-Square

- Corrélation de Pearson : quantifie la dépendance linéaire entre deux variables continues X et Y.
- LDA : Linear discriminant analysis (LA) est utilisée pour trouver une combinaison linéaire de caractéristiques qui caractérise ou sépare deux ou plusieurs classes d'une variable catégorielle.
- ANOVA : ANOVA (Analyse de la variance) fournit un test statistique permettant de déterminer si les moyennes de plusieurs groupes sont égales ou non.
- Chi-2 : Il s'agit d'un test statistique appliqué aux groupes de caractéristiques catégorielles pour évaluer la probabilité de corrélation ou d'association entre eux en utilisant leur distribution de fréquence.

Feature selection

Méthode par wrapper

Méthodes wrapper



- Un sous-ensemble de features est utilisé pour entraîner un modèle de ML.
- Sur la base des inférences tirées du modèle précédent, des features sont ajoutées ou supprimées du sous-ensemble.
- Le problème est essentiellement réduit à un problème de recherche. Ces méthodes sont généralement très coûteuses en termes de calcul.

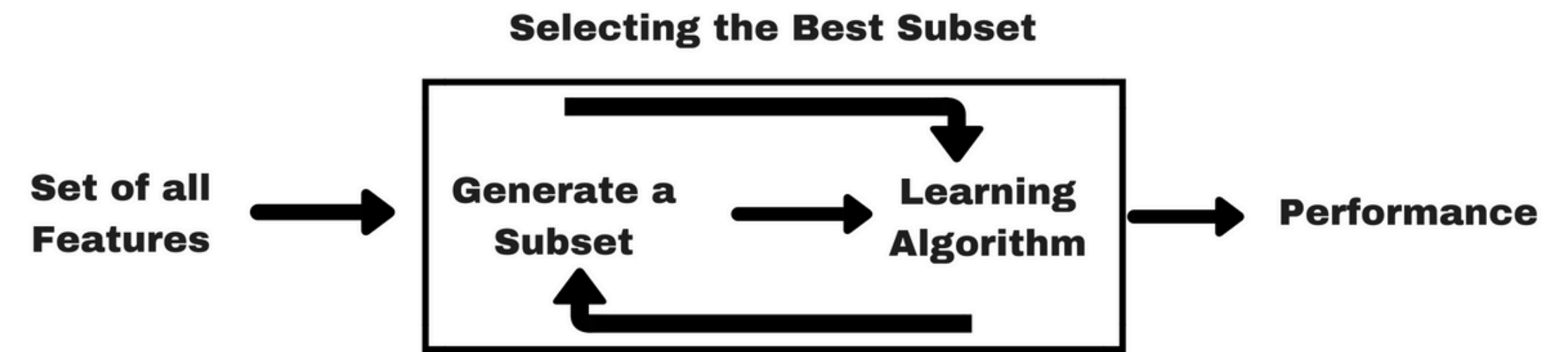
Boruta

- *Boruta is an all relevant feature selection method, while most other are minimal optimal; this means it tries to find all features carrying information usable for prediction, rather than finding a possibly compact subset of features on which some classifier has a minimal error.*

Miron B. Kursa

- Kursa, M.B. and Rudnicki, W.R., 2010. Feature selection with the Boruta package. Journal of statistical software, 36, pp.1-13.
<https://www.jstatsoft.org/article/view/v036i11>

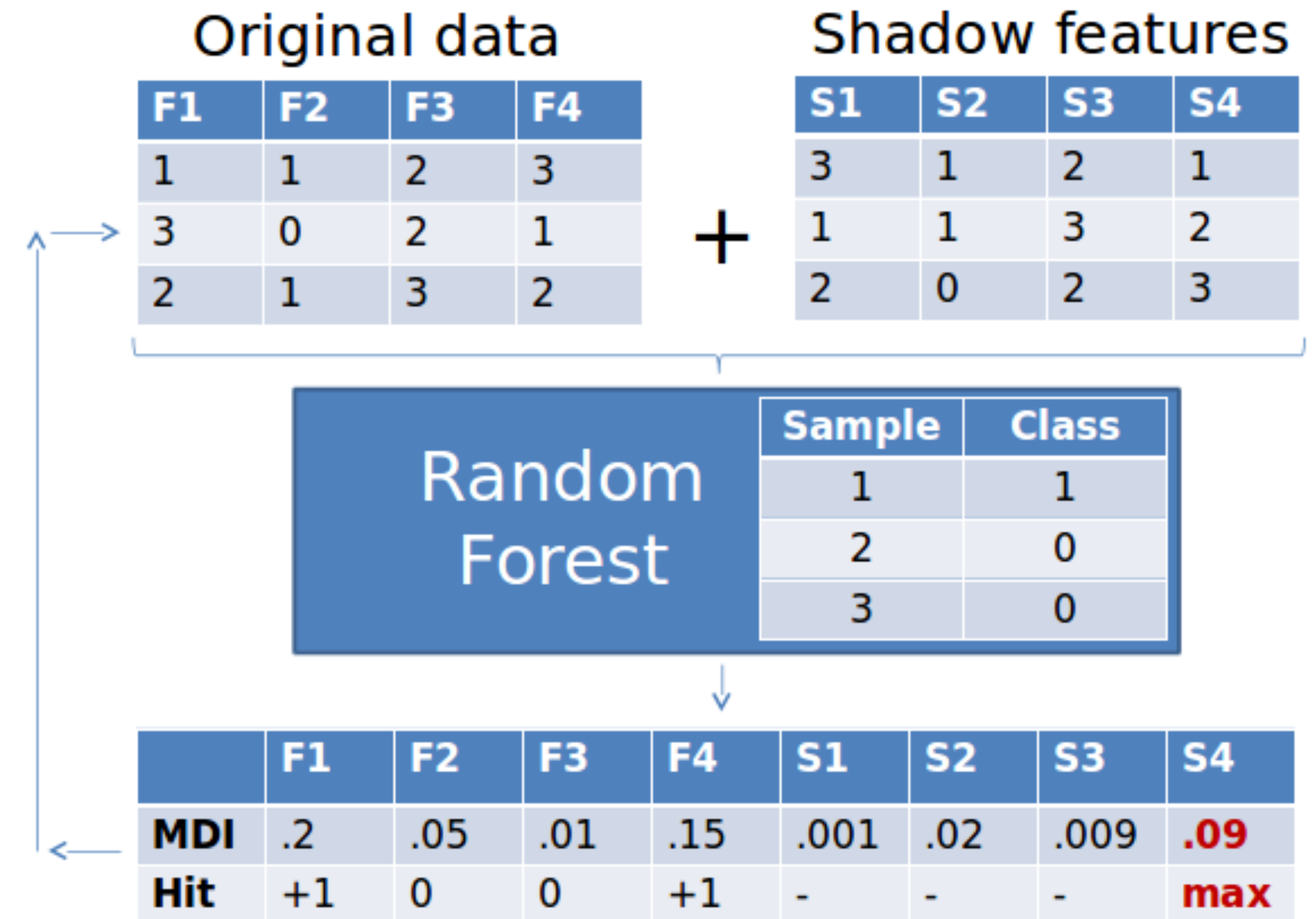
Algorithme de Boruta



1. Extension du dataset en copiant toutes les features (le nombre de features double).
2. Donner des valeurs aléatoires à ces colonnes ajoutées (*shadow features*).
3. Entraîner un classifieur Random Forest sur le dataset étendu et évaluer l'impact de chaque feature (Mean Decrease Accuracy, Z score...).
4. Trouver la shadow feature avec le plus haut score et donner 1 point à toutes les features avec un score supérieur. Répéter cette procédure plusieurs fois.
5. Retirer du dataset les features qui n'ont pas reçu de point en N itérations (les noter **rejetés**) et celles qui font un bien meilleur score que la meilleure shadow feature (les noter **confirmés**).
Et recommencer l'étape 4.
6. S'arrêter quand toutes les features ont été rejetés ou confirmés, ou après un certain nombre d'itérations.

Algorithme de Boruta

- Dans l'algorithme de RandomForest, l'importance des features est calculé par la moyenne de leur *baisse d'impureté* (mean decrease impurity), qui correspond au coefficient de Gini.



- Accessible par l'attribut `feature_importances_` du classifieur

	F1	F2	F3	F4	S1	S2	S3	S4
MDI	.2	.05	.01	.15	.001	.02	.009	.09
Hit	+1	0	0	+1	-	-	-	max
Hit	+1	+1	0	0	-	max	-	-
Hit	+1	0	0	+1	-	-	-	max

Différences entre filtrage et wrapper

Filtrage	Wrapper
Mesurent la pertinence des features par leur corrélation avec des variables dépendantes	Mesurent l'utilité d'un sous-ensemble de features en formant un modèle sur ces dernières
Beaucoup plus rapides car elles n'impliquent pas l'entraînement des modèles	Très coûteuses en termes de calcul
Utilisent des méthodes statistiques pour l'évaluation d'un sous-ensemble de caractéristiques	Utilisent la validation croisée
Peuvent échouer à trouver le meilleur sous-ensemble de caractéristiques dans de nombreuses occasions	Fournit toujours le meilleur sous-ensemble de caractéristiques
Sous-ensemble obtenu peu enclin au sur-apprentissage	Sous-ensemble obtenu potentiellement enclin au sur-apprentissage

Explicabilité en Machine Learning (XAI)

Des modèles interprétables et d'autres non

Glass Box vs Black Box

- Nous avons vu beaucoup d'algorithmes de Machine Learning
 - Regression linéaire
 - KNN
 - SVM
 - Arbres de décision
 - Random Forest
 - Gradient boosting
 - Réseaux de neurones

Des modèles interprétables et d'autres non

Glass Box vs Black Box

- Nous avons vu beaucoup d'algorithmes de Machine Learning
 - Regression linéaire
 - KNN
 - SVM
 - Arbres de décision
 - Random Forest
 - Gradient boosting
 - Réseaux de neurones

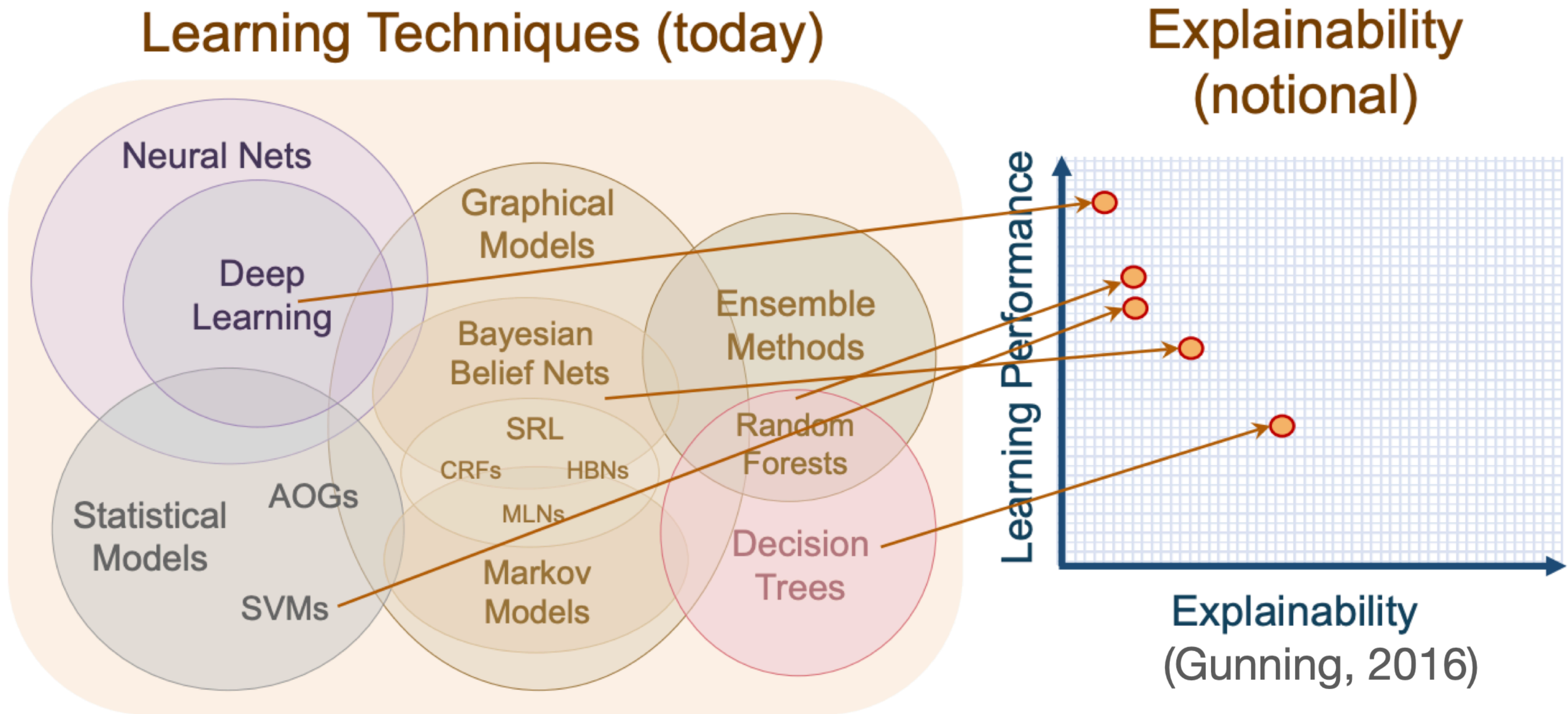
Des modèles interprétables et d'autres non

Glass Box vs Black Box

- Nous avons vu beaucoup d'algorithmes de Machine Learning
 - Regression linéaire
 - KNN
 - SVM
 - Arbres de décision
 - Random Forest
 - Gradient boosting
 - Réseaux de neurones
- | | | |
|----------|-----------------------------|------------|
| Simple | Interprétable | Peu précis |
| Complexe | Difficilement interprétable | Précis |

Des modèles interprétables et d'autres non

Glass Box vs Black Box



Interprétation des modèles

Interprétation globale vs locale

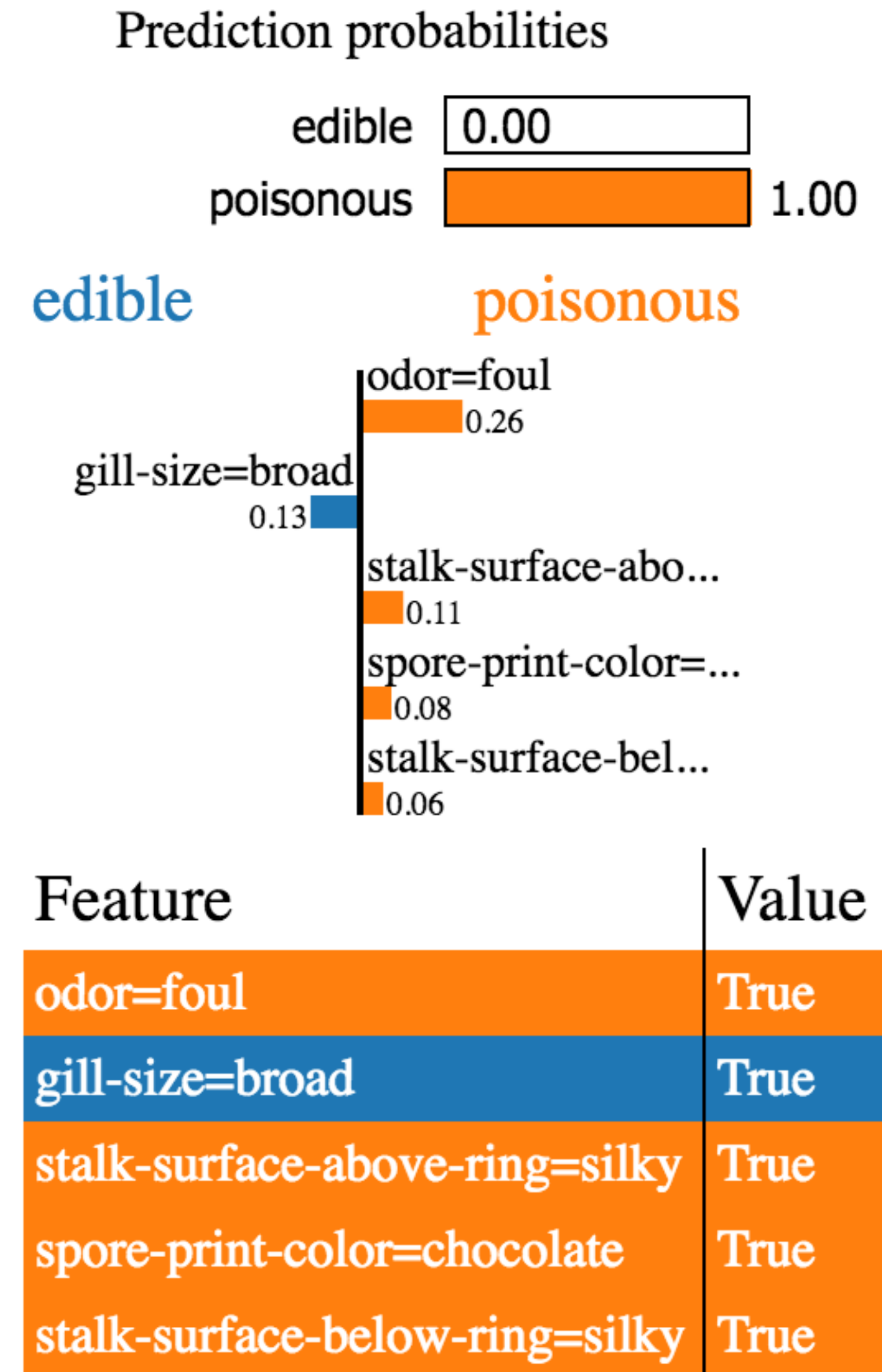
- Interprétation locale
 - Aide à la compréhension de la décision prise pour 1 observation
 - Explication des prédictions individuelles
 - Compréhension du comportement du modèle dans son voisinage local
- Interprétation globale
 - Aide à la compréhension de la décision au niveau structurel du modèle
 - Explication du comportement global des prédictions
 - Permet d'appréhender l'aptitude du modèle à être déployé

Interprétation locale

LIME

Local Interpretable Model-agnostic Explanations

- Méthode indépendante de l'algorithme de ML
- Choix d'une instance à expliquer
- Calcul de l'impact de permutations des entrées sur les prédictions obtenues
- Entraînement d'une régression linéaire (ou tout autre méthode interprétable) pour pondérer les perturbations par rapport à la distance de l'instance originale



SHAP

SHapley Additive exPlanations

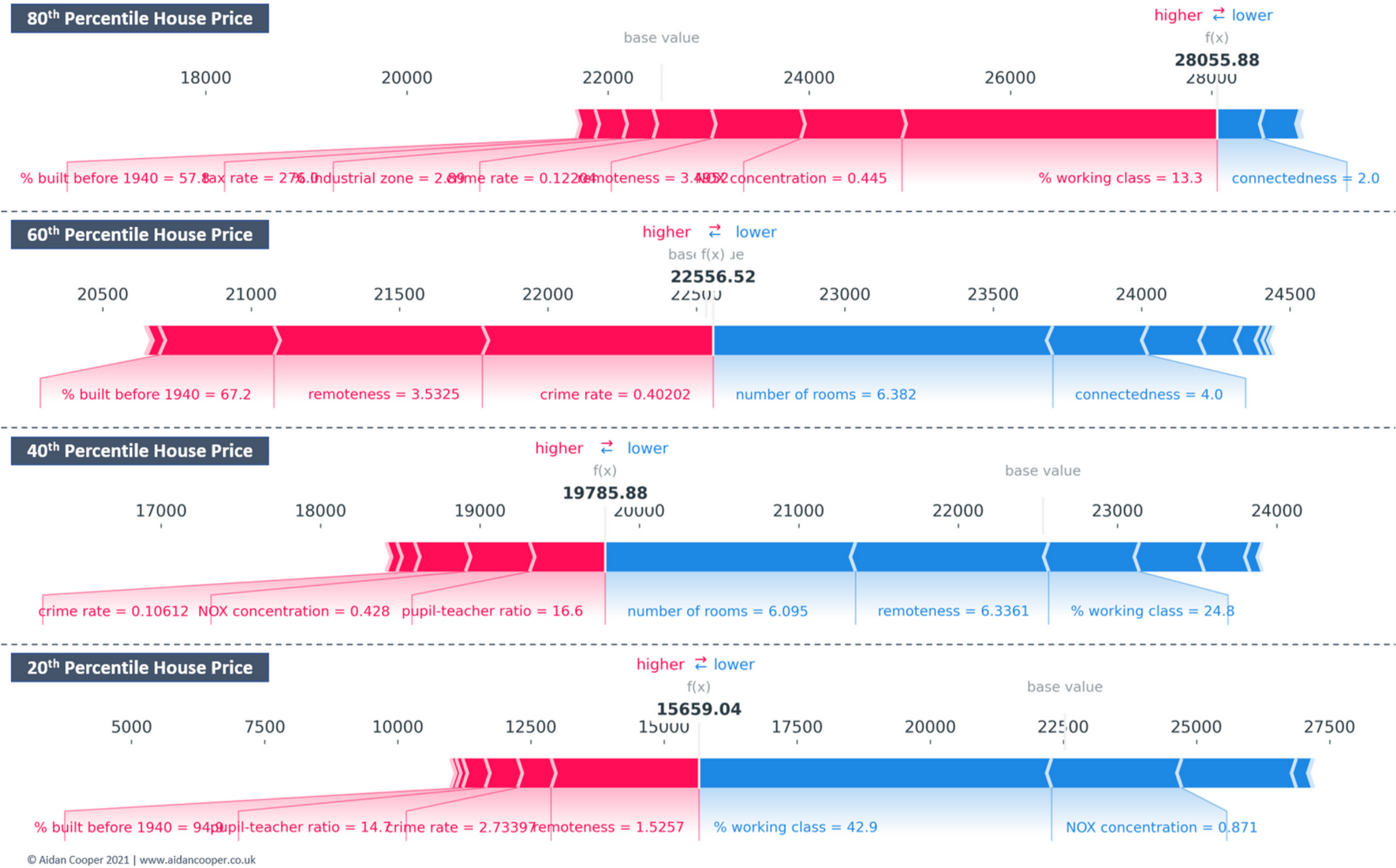
- Algorithme basé sur la théorie des jeux
 - Le jeu est la prédiction faite par le modèle pour une observation donnée
 - Les joueurs sont les caractéristiques de l'observation
- Calcul de la contribution de chaque caractéristique par rapport aux autres
- Plus précisément, les SHAP values répondent à la question
Combien une variable à influencer la prédiction par rapport à la prédiction de la valeur moyenne du dataset (baseline) ?

SHAP

SHapley Additive exPlanations

- Boston housing dataset :
Prédire le prix d'un logement à Boston à partir de 12 indicateurs sur le quartier

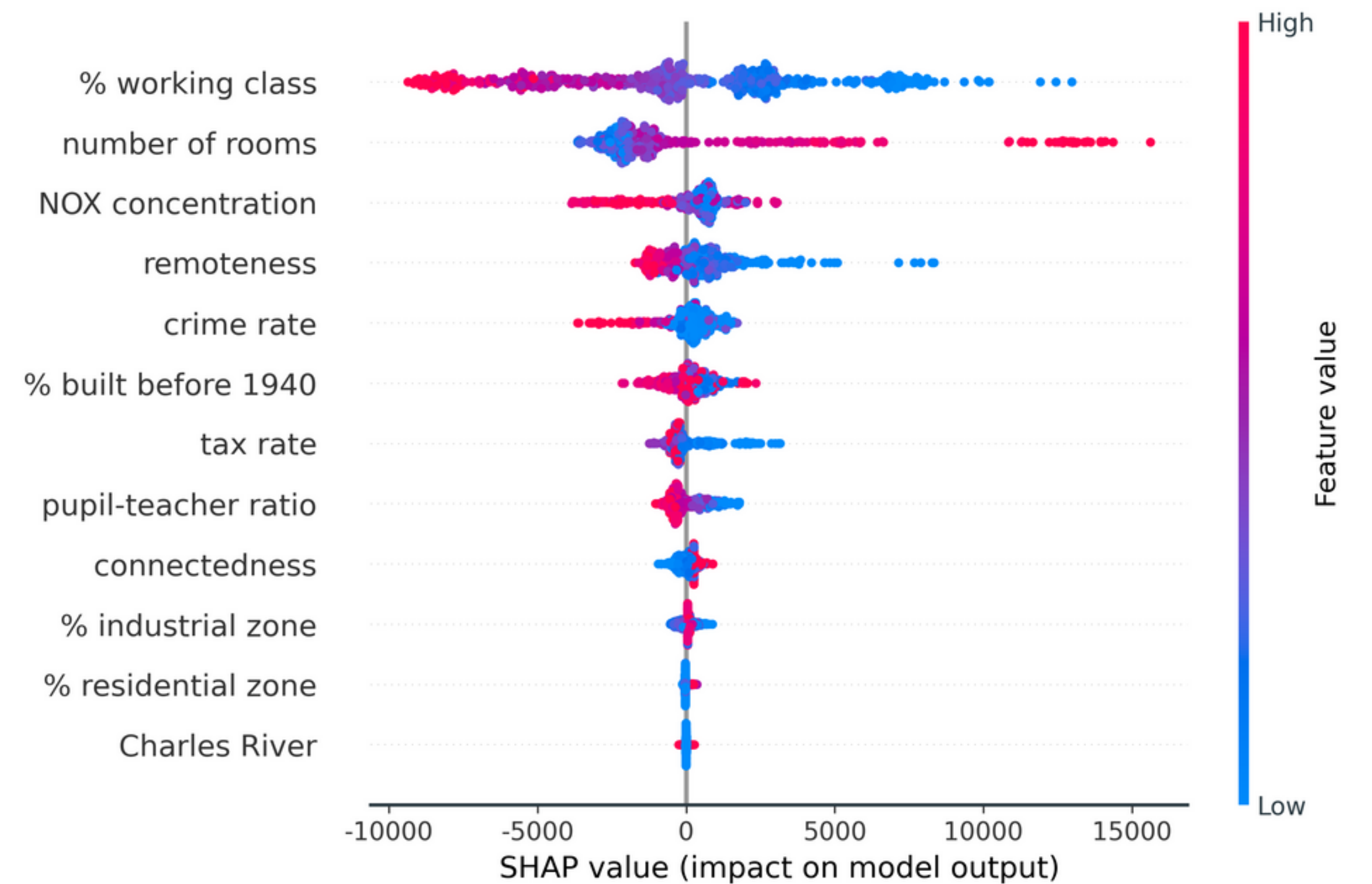
Variable Name	Description
crime rate	Per capita crime rate in the town.
% residential zone	Percentage of land zoned for residential use.
% industrial zone	Percentage of land zoned for industrial use.
Charles River	1 if the house borders the Charles River; 0 otherwise.
NOX concentration	Nitric oxides concentration (parts per 10 million).
number of rooms	The average number of rooms per house in the housing unit.
% built before 1940	The proportion of houses built prior to 1940 in the unit.
remoteness	A measure of how far the housing is from employment centres (higher is more remote).
connectedness	A measure of how good the local road connections are (higher is more connected).
tax rate	Property tax rate per \$10,000 of house value.
pupil-teacher ratio	The pupil:teacher ratio of the town.
% working class	Percentage of the population that is working class.



Interprétation globale

SHapley Additive exPlanations

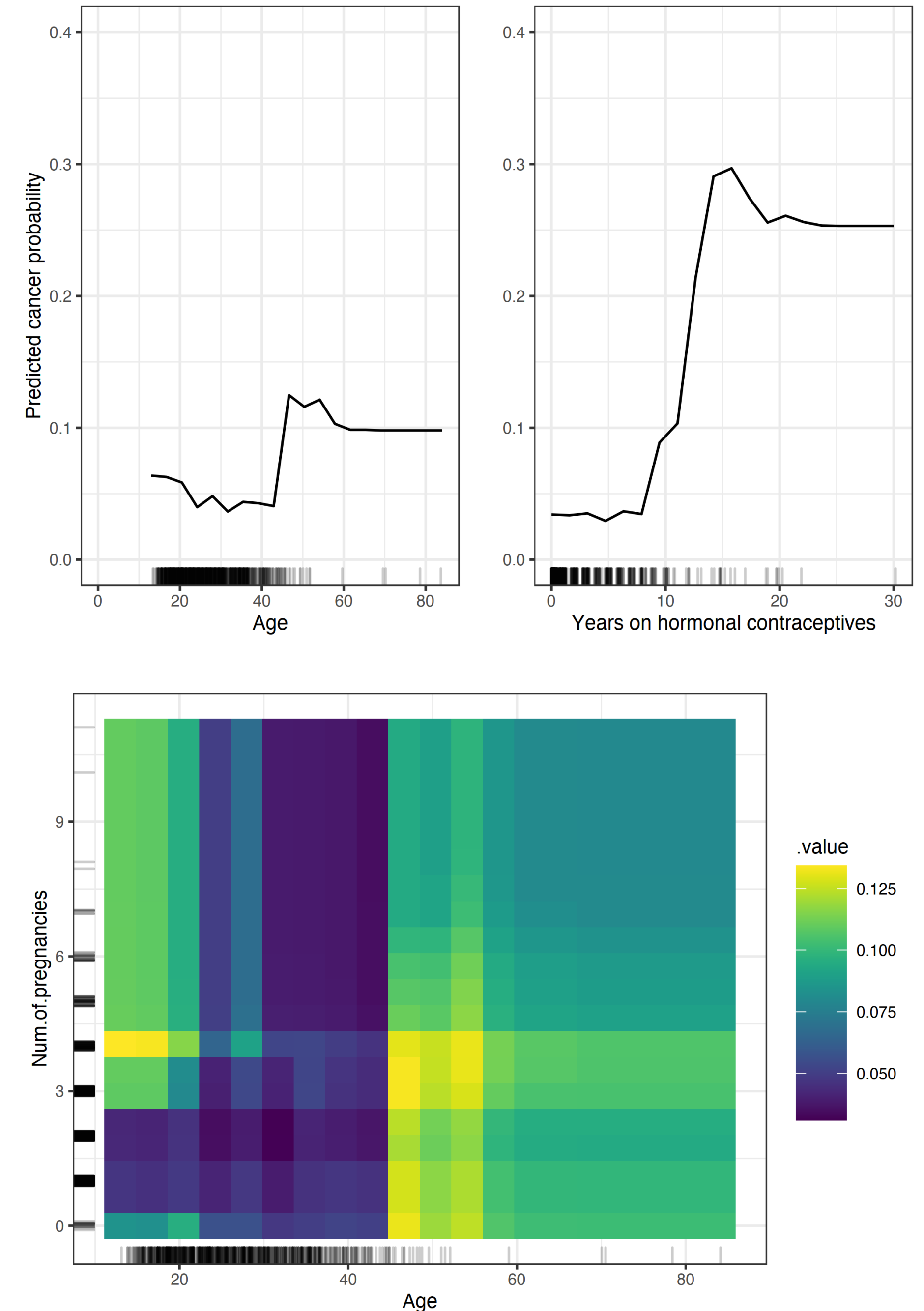
- Peut-être utilisé pour faire de l'explicabilité globale en représentant toutes les explications locales



PDP

Partial Dependency Plot

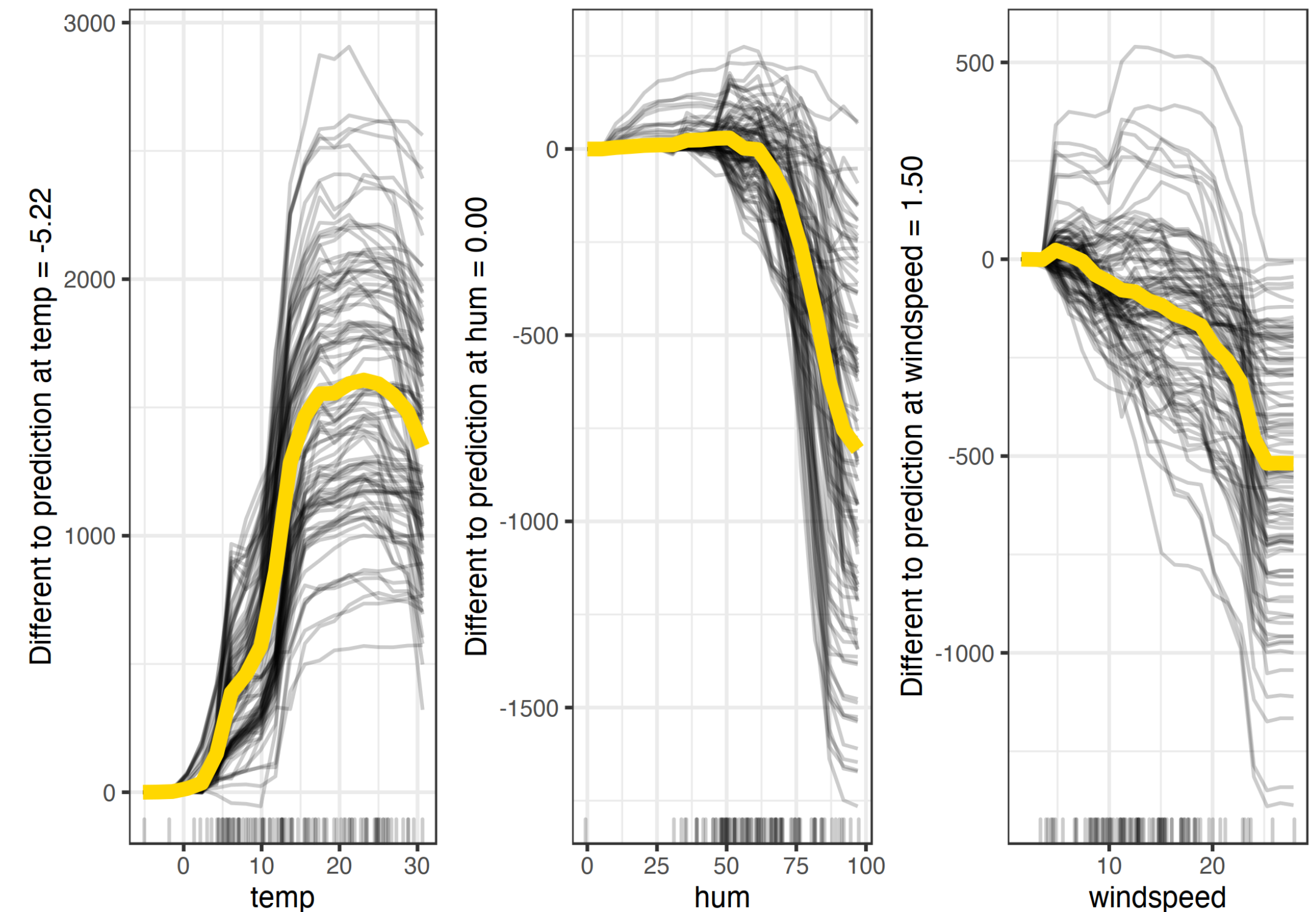
- Effet marginal d'une caractéristique sur la prédiction
- Plus précisément, moyenne des prédictions faites pour chaque valeur de la caractéristique étudiée
- Limité à l'étude d'une ou deux caractéristiques en même temps (nécessité d'indépendance entre variables)
- Ne montre pas l'hétérogénéité des données



ICE

Individual Conditional Expectation

- Visualisation de la dépendance entre une variable et la prédiction
- Montre l'impact de l'évolution de la valeur d'une caractéristique sur la prédiction pour chaque instance du dataset
- Affiche l'hétérogénéité des données
- Limité à une seule caractéristique à la fois



Pourquoi c'est important ?

- Technique & Modèles
 - Améliorer les modèles
 - Meilleur contrôle de modèles
- Décisionnaires
 - Décision équitable
 - Réduction des discriminations
- **Confiance dans les modèles et dans l'IA**

Demo Explicabilité